Report: Optimising NYC Taxi Operations

Include your visualisations, analysis, results, insights, and outcomes. Explain your methodology and approach to the tasks. Add your conclusions to the sections.

1. Data Preparation

1.1. Loading the dataset

1.1.1. Sample the data and combine the files

All the file were read one by one. For each file we did following steps:

a. Read the file
b. For each date loop through each hour, then for each hour collect 5% sampled data
c. Concat 5% sampled data to main dataframe which is initialize to store all samples.

```
df = pd.read_parquet("/Users/inirala/Documents/Job Lookup/AIML LJMU/2. Data Analysis
and Exploration'/EDA_assignment/data_NYC_taxi/2023-1.parquet")
# from google.colab import drive
#drive.mount('/content/drive')
import os
os.chdir('/Users/inirala/Documents/Job Lookup/AIML LJMU/2. Data Analysis and
Exploration\'/EDA_assignment/data_NYC_taxi/')
file_list = os.listdir()
print(file_list)
# Take a small percentage of entries from each hour of every date.
# Iterating through the monthly data:
#   read a month file -> day -> hour: append sampled data -> move to next hour -> move
to next day after 24 hours -> move to next month file
# Create a single dataframe for the year combining all the monthly data

# Select the folder having data files
import os

# Select the folder having data files
os.chdir('/Users/inirala/Documents/Job Lookup/AIML LJMU/2. Data Analysis and
Exploration\'/EDA_assignment/data_NYC_taxi/')

# Create a list of all the twelve files to read
file_list = os.listdir()
```

Cisco Confidential

```
# initialise an empty dataframe
df = pd.DataFrame()


# iterate through the list of files and sample one by one:
for file_name in file_list:
    try:
        # file path for the current file
        file_path = os.path.join(os.getcwd(), file_name)

        # Reading the current file
        df_current = pd.read_parquet(file_path)
        df_current['tpep_pickup_datetime'] =
pd.to_datetime(df_current['tpep_pickup_datetime'])


        # We will store the sampled data for the current date in this df by appending
the sampled data from each hour to this
        # After completing iteration through each date, we will append this data to
the final dataframe.
        sampled_data = pd.DataFrame()

        # Loop through dates and then loop through every hour of each date
        for date, df_date in
df_current.groupby(df_current['tpep_pickup_datetime'].dt.date):

            # Iterate through each hour of the selected date
            for hour, df_hour in
df_date.groupby(df_date['tpep_pickup_datetime'].dt.hour):
                # Sample 5% of the hourly data randomly
                df_hour_sampled = df_hour.sample(frac=0.05, random_state=42)

                # add data of this hour to the dataframe
                sampled_data = pd.concat([sampled_data, df_hour_sampled],
ignore_index=True)

        # Concatenate the sampled data of all the dates to a single dataframe
        df = pd.concat([df, sampled_data], ignore_index=True) # we initialised this
empty DF earlier

    except Exception as e:
        print(f"Error reading file {file_name}: {e}")
```

Cisco Confidential

2. Data Cleaning

2.1. Fixing Columns

2.1.1. Fix the index

a. Reset the dataframe index, index start with 0.

b. Drop column store_and_fwd_flag as it seemed to me unnecessary in analysis.

```
# Fix the index and drop any columns that are not needed

df.reset_index(drop=True, inplace=True)
df.drop(columns=['store_and_fwd_flag'], inplace=True)
```

2.1.2. Combine the two airport_fee columns

a. Store sum of **Airport_fee** column and **airport_fee** column to **airport_fee** column after replacing each null value with 0.

b. Drop **Airport_fee** column

```
# Combine the two airport fee columns
df["airport_fee"] = df["Airport_fee"].fillna(0) + df["airport_fee"].fillna(0)
df = df.drop(columns=["Airport_fee"])
```

2.2. Handling Missing Values

2.2.1. Find the proportion of missing values in each column

a. Found the proportion of missing values by executing **df.isnull().mean()**

```

# Find the proportion of missing values in each column
df.isnull().mean()
```

2.2.2. Handling missing values in passenger_count

a. The mode is the passenger count that appears most often in the data. So, most probable value for missing values is the mode value. So,

Cisco Confidential

each missing value is replaced with that common count.

```
# Display the rows with null values
# Impute NaN values in 'passenger_count'
df["passenger_count"] =
df["passenger_count"].fillna(df["passenger_count"].mode()[0])
```

2.2.3. Handle missing values in RatecodeID

a. The mode is the RateCodeID that appears most often in the data. So, each missing value is replaced with that common count.

```
# Fix missing values in 'RatecodeID'
df["RatecodeID"] = df["RatecodeID"].fillna(df["RatecodeID"].mode()[0])
```

2.2.4. Impute NaN in congestion_surcharge

a. The mode is the congestion_surcharge that appears most often in the data. So, each missing value is replaced with that common count.

```
# handle null values in congestion_surcharge
df["congestion_surcharge"] =
df["congestion_surcharge"].fillna(df["congestion_surcharge"].mode()[0])
```

2.3. Handling Outliers and Standardising Values

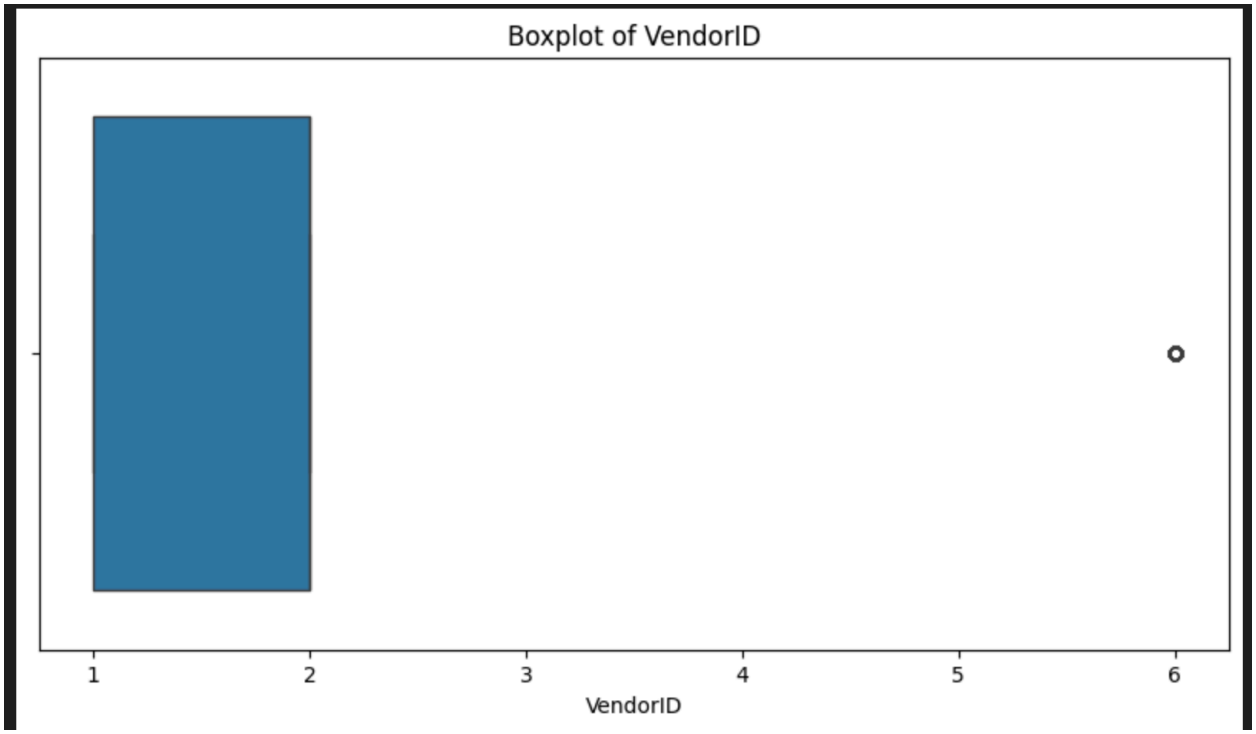
Drawing boxplot for all the columns with datatype value ‘float64’ or ‘int64’ to identify outliers

```
# Describe the data and check if there are any potential outliers present
# Check for potential out of place values in various columns
df.describe()

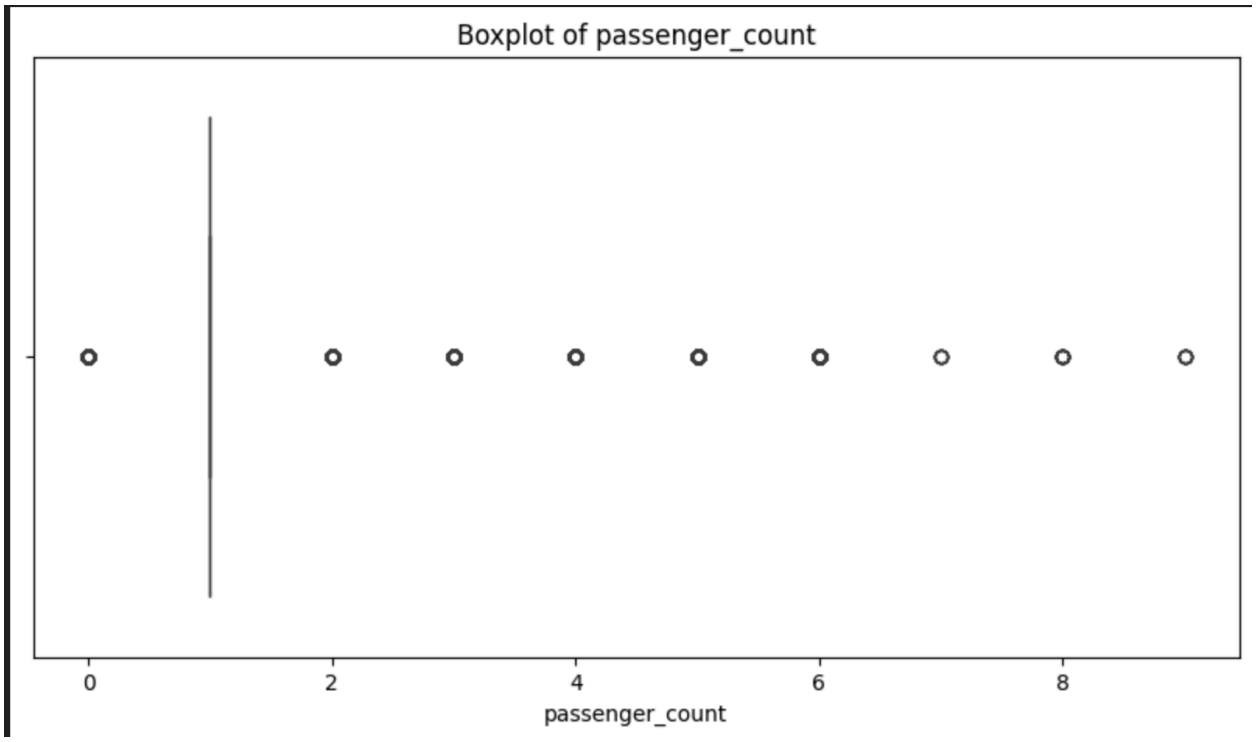
# Visualize potential outliers using boxplots
import matplotlib.pyplot as plt
import seaborn as sns
```

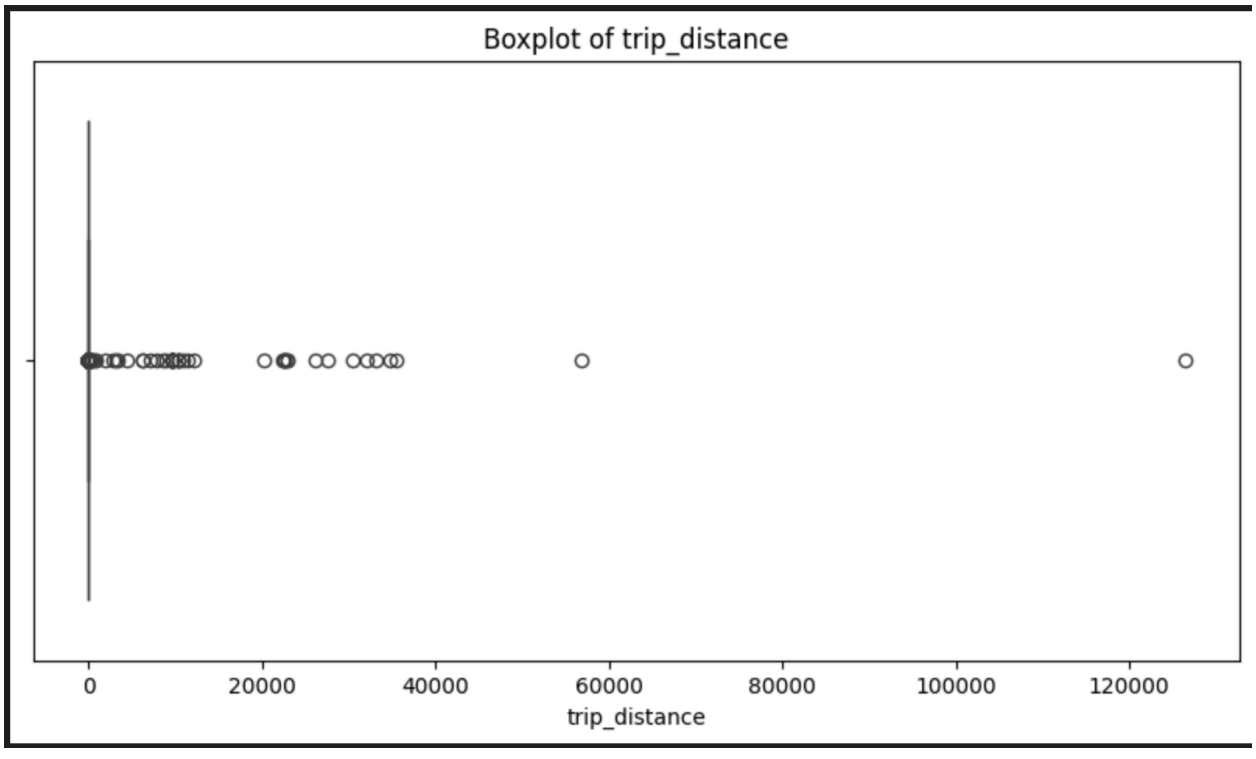
Cisco Confidential

```
for column in df.select_dtypes(include=['float64', 'int64']).columns:
    plt.figure(figsize=(10, 5))
    sns.boxplot(x=df[column])
    plt.title(f'Boxplot of {column}')
    plt.show()
```

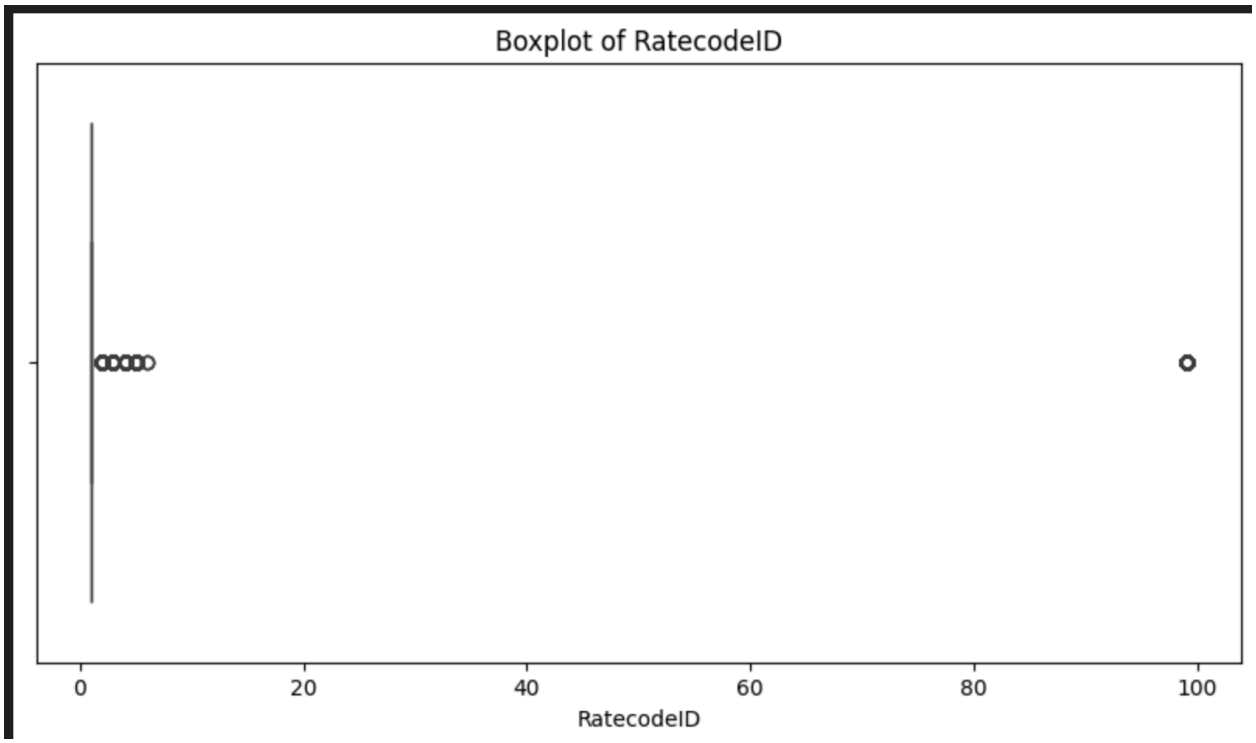


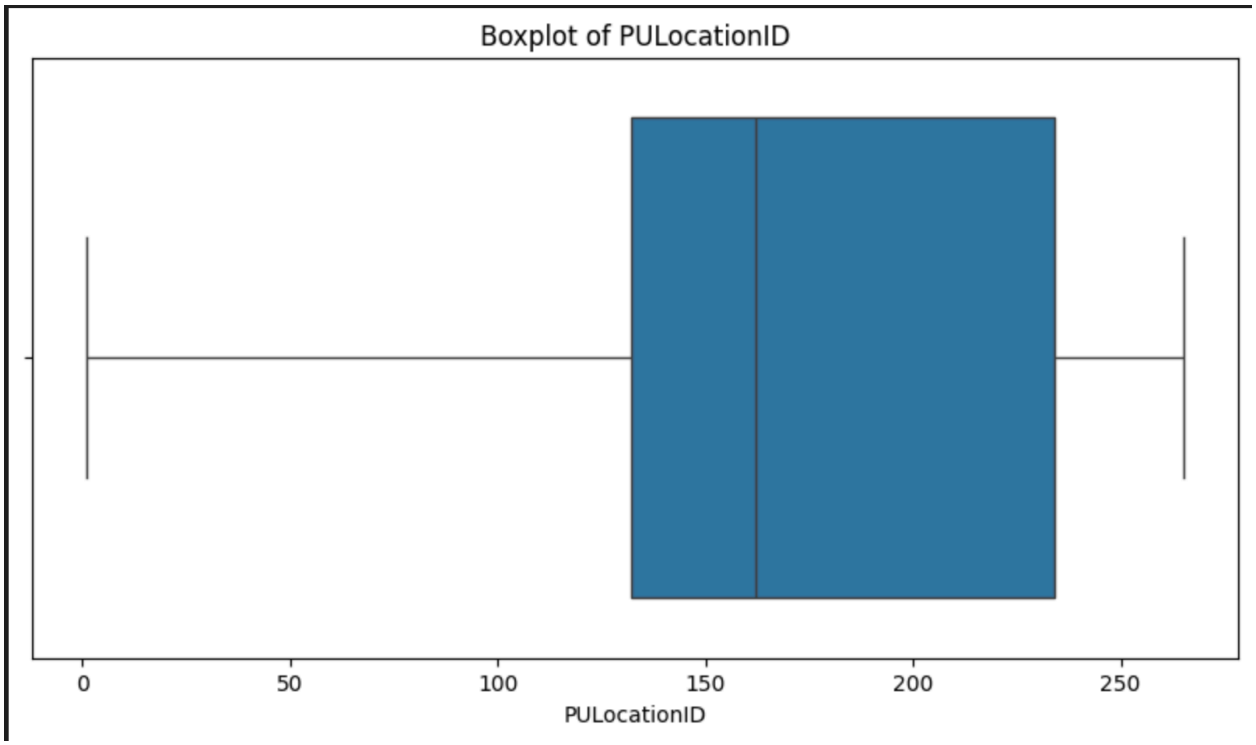
Cisco Confidential



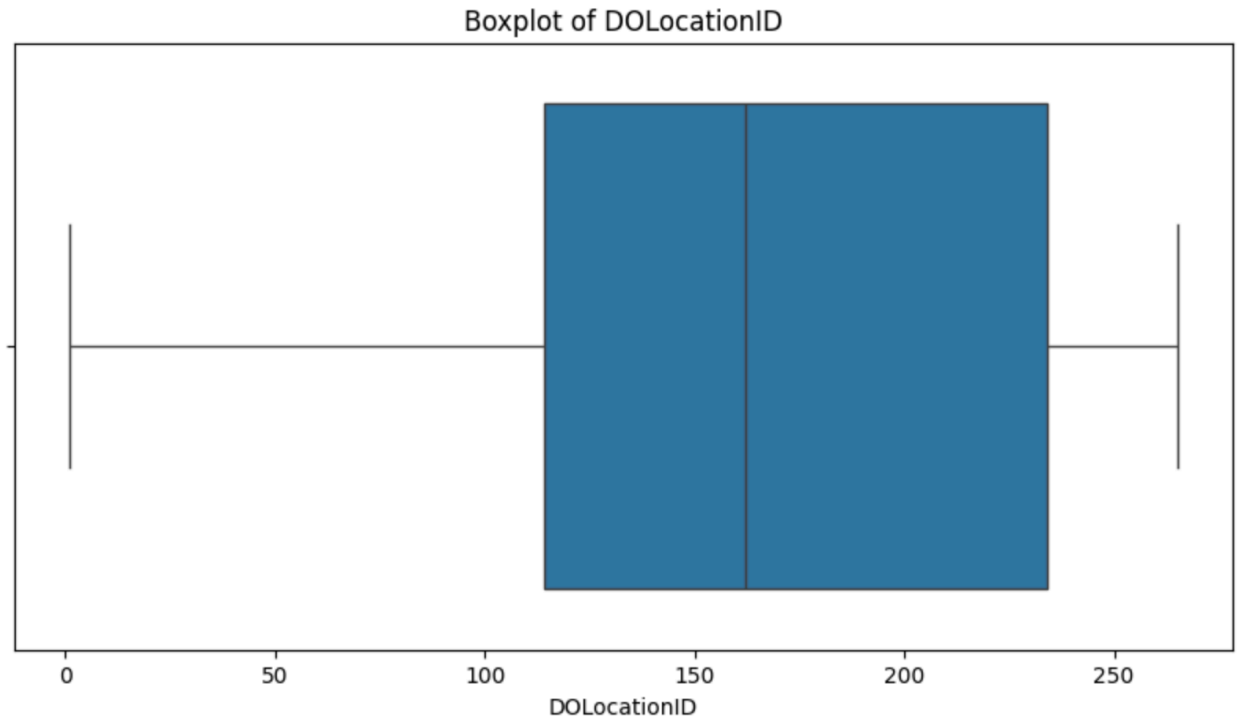


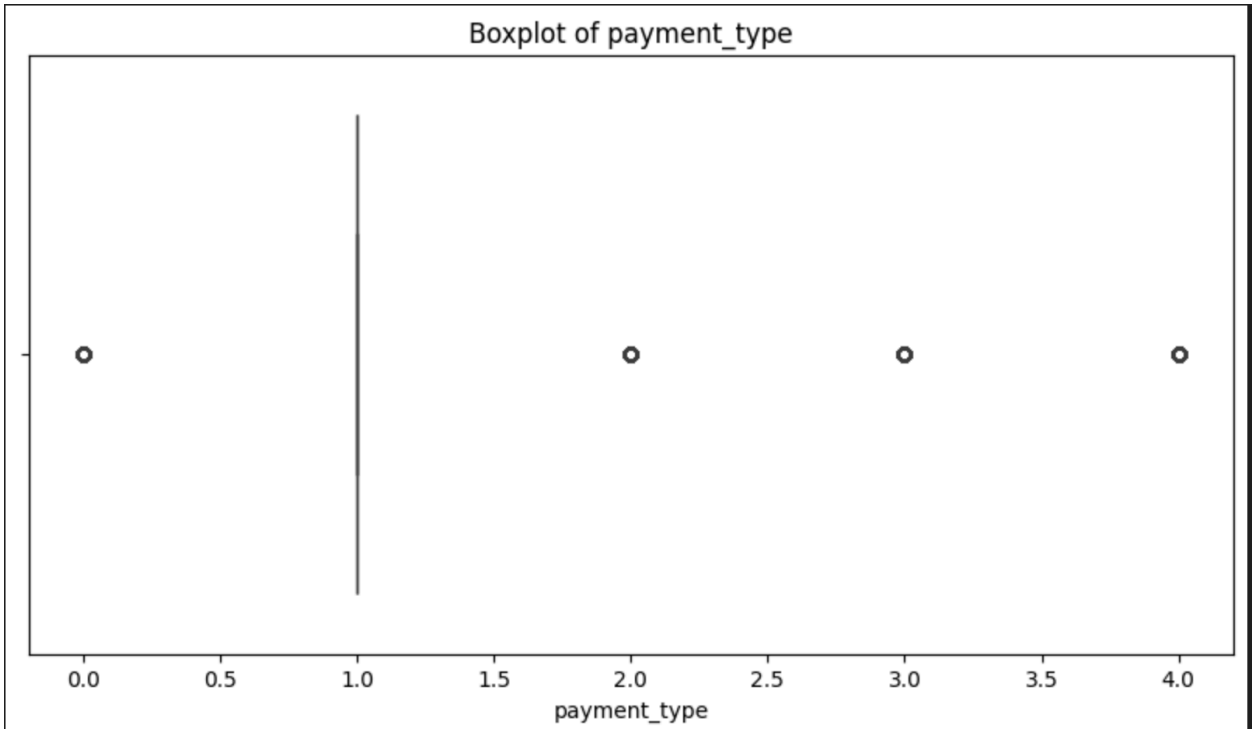
Cisco Confidential



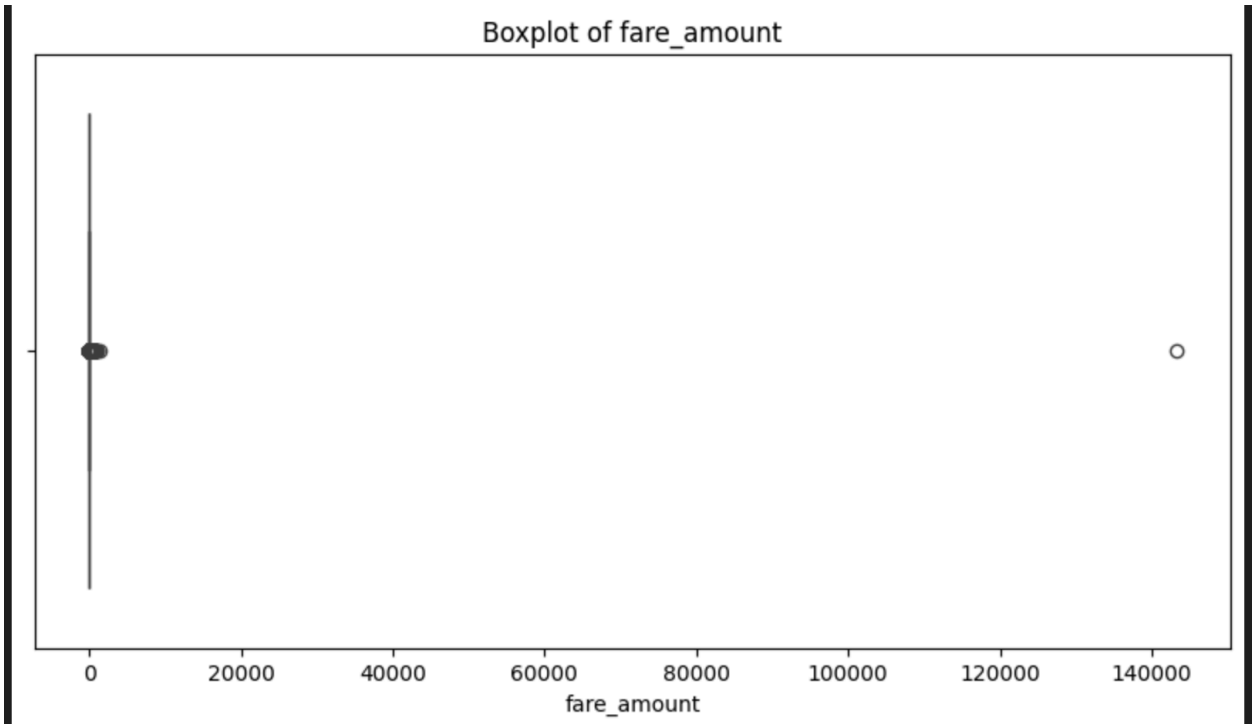


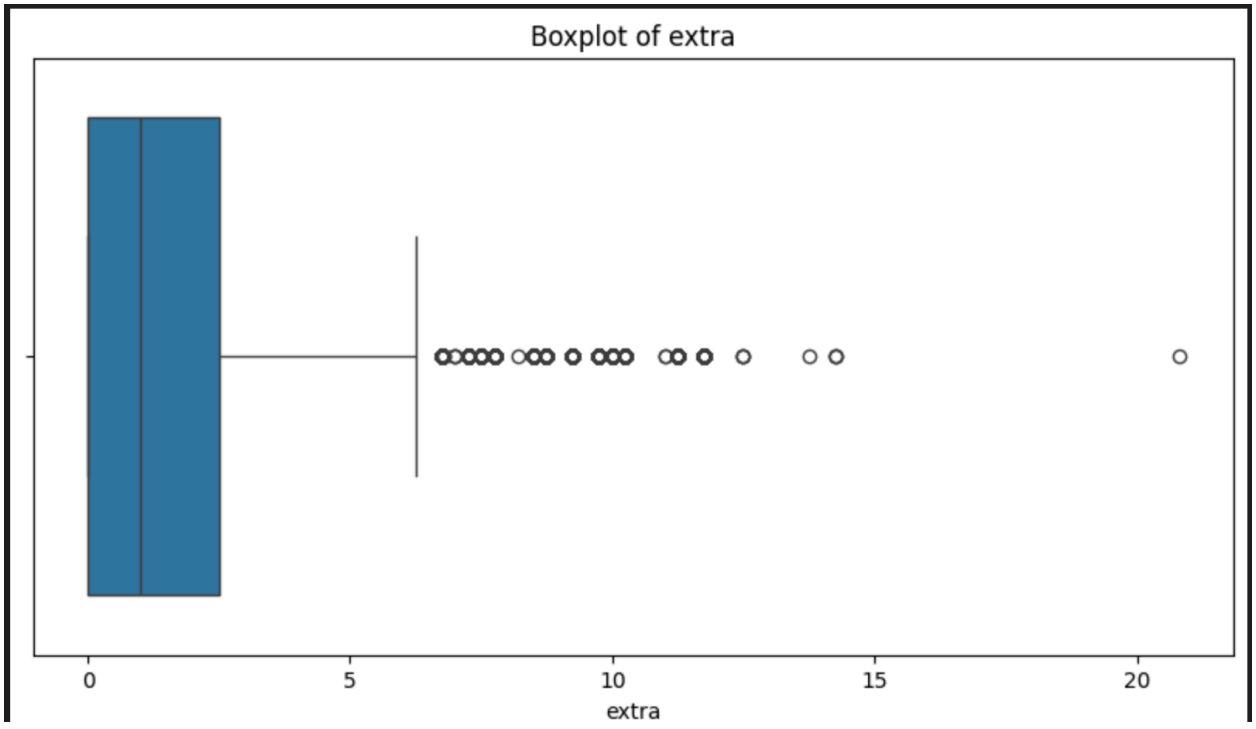
Cisco Confidential



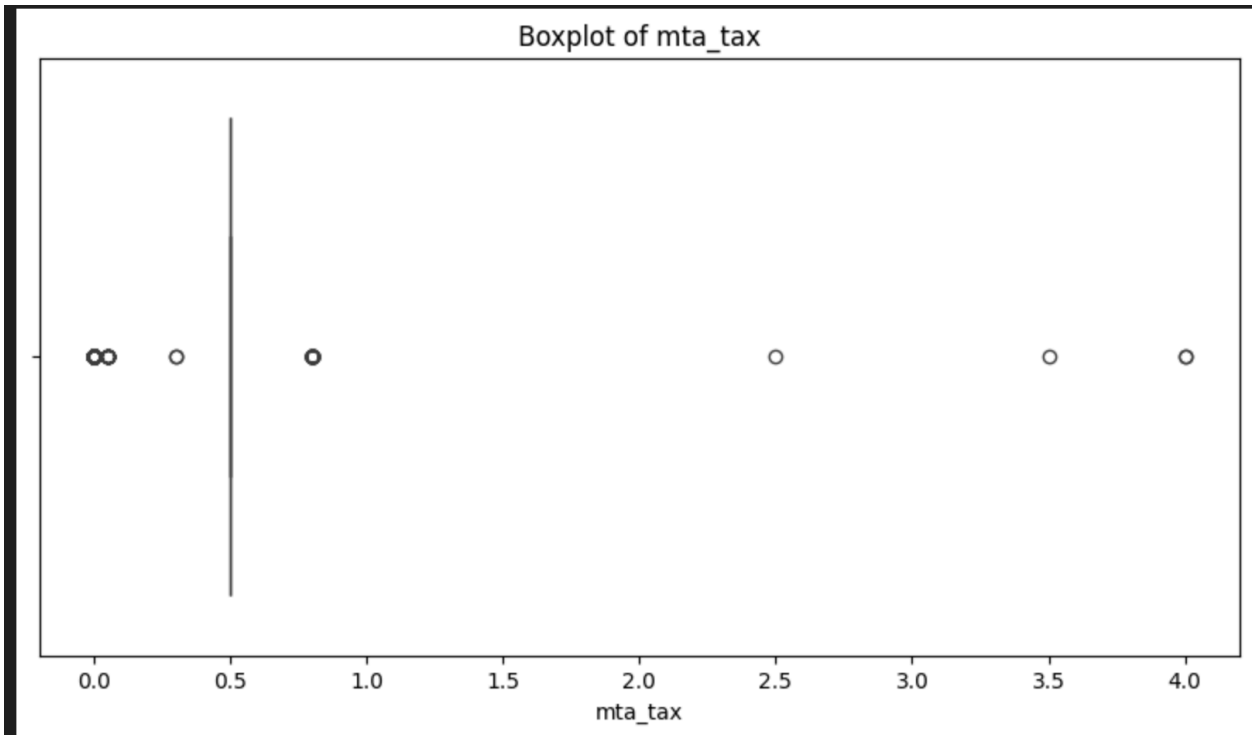


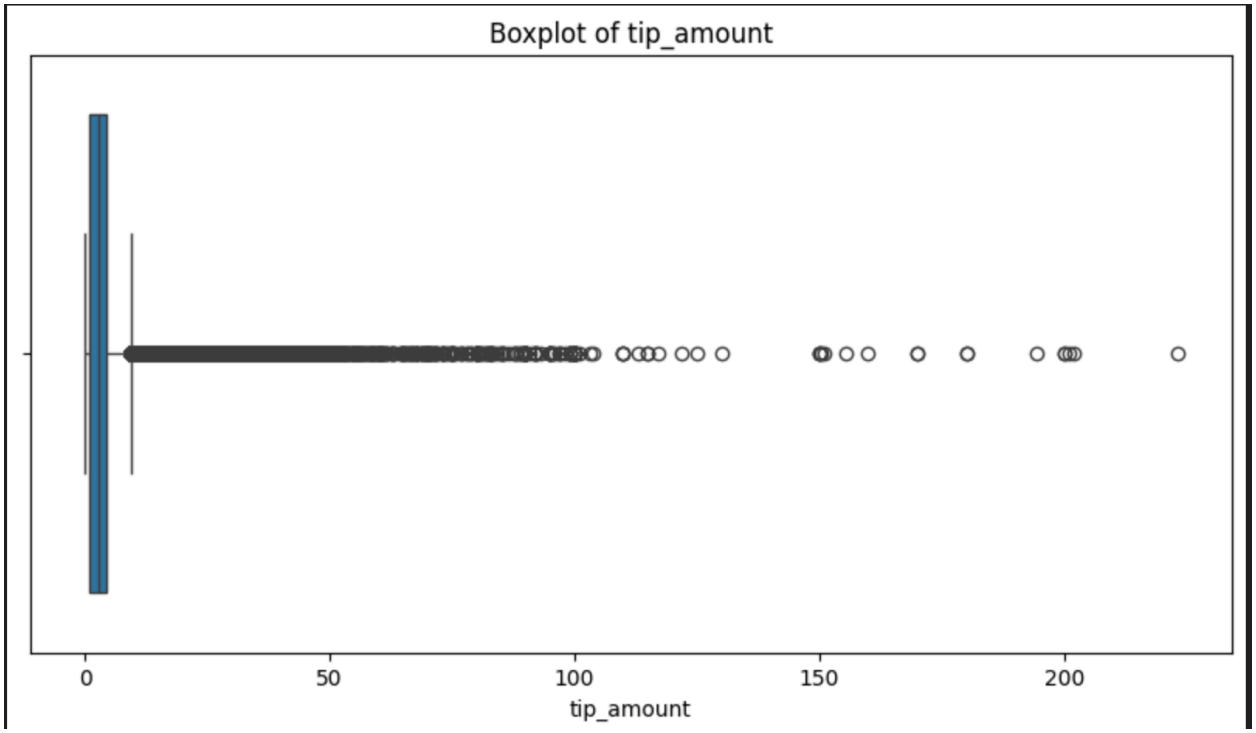
Cisco Confidential



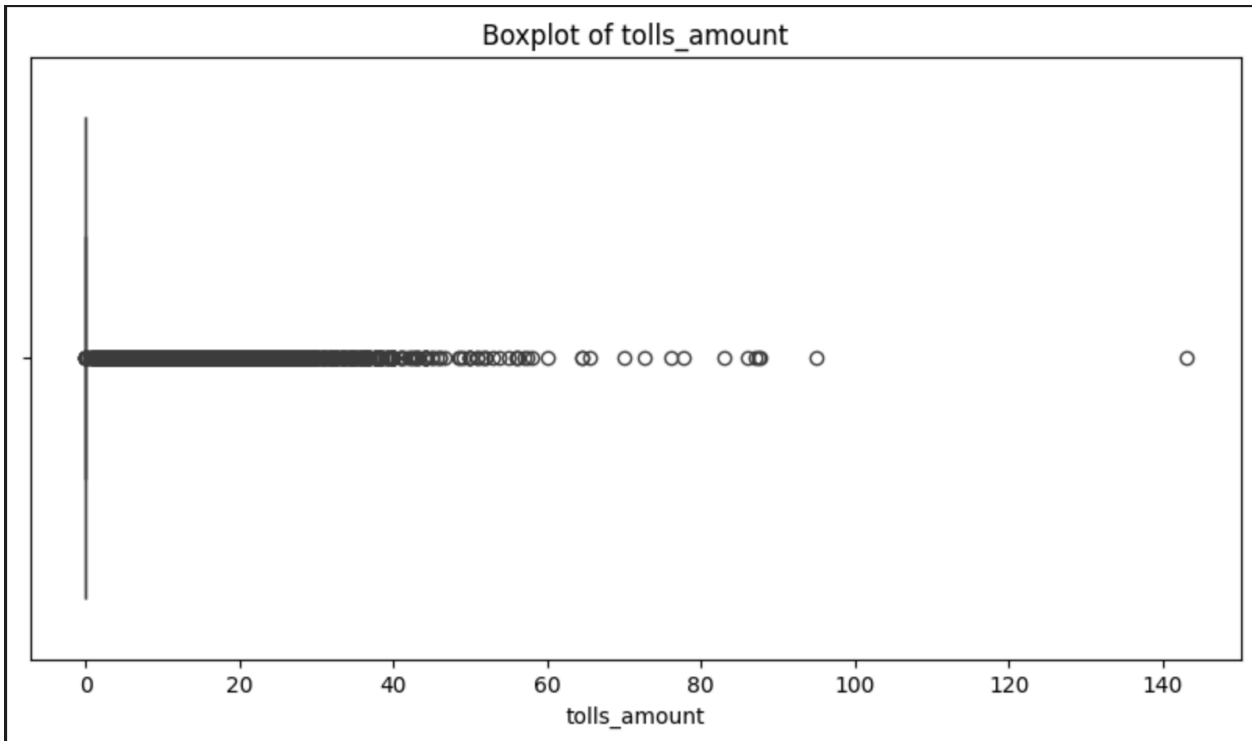


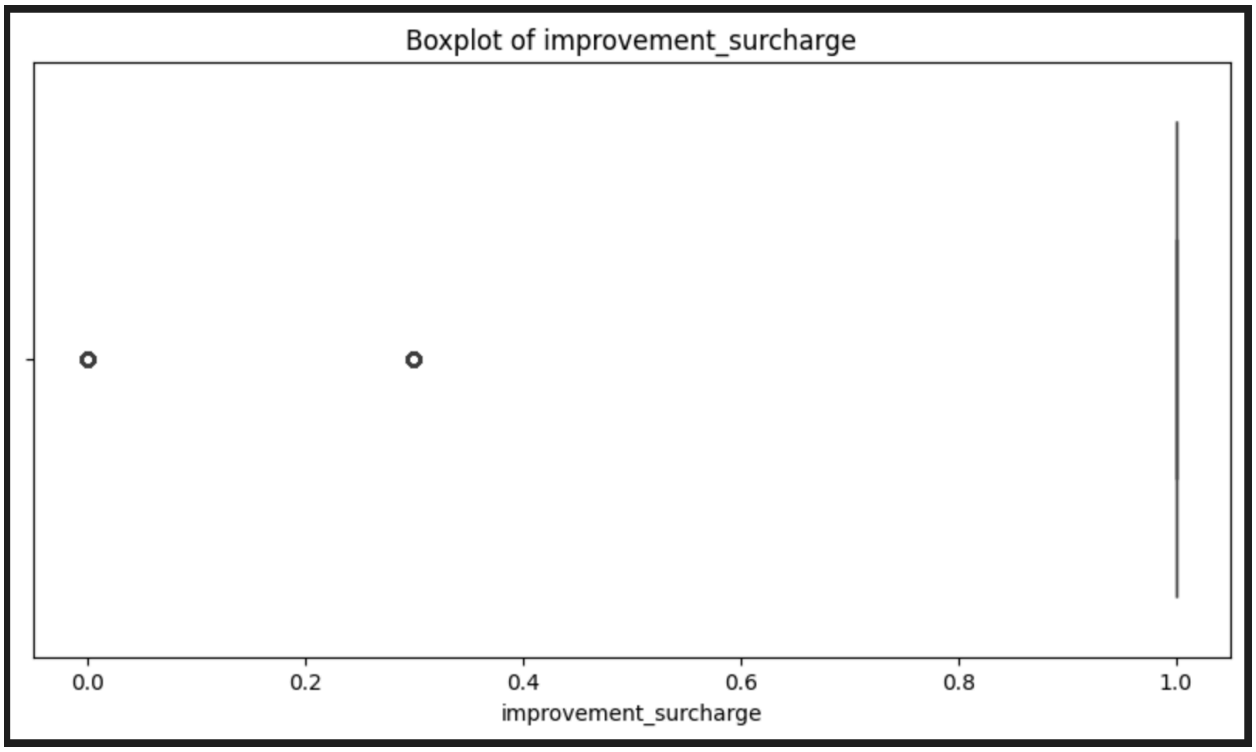
Cisco Confidential



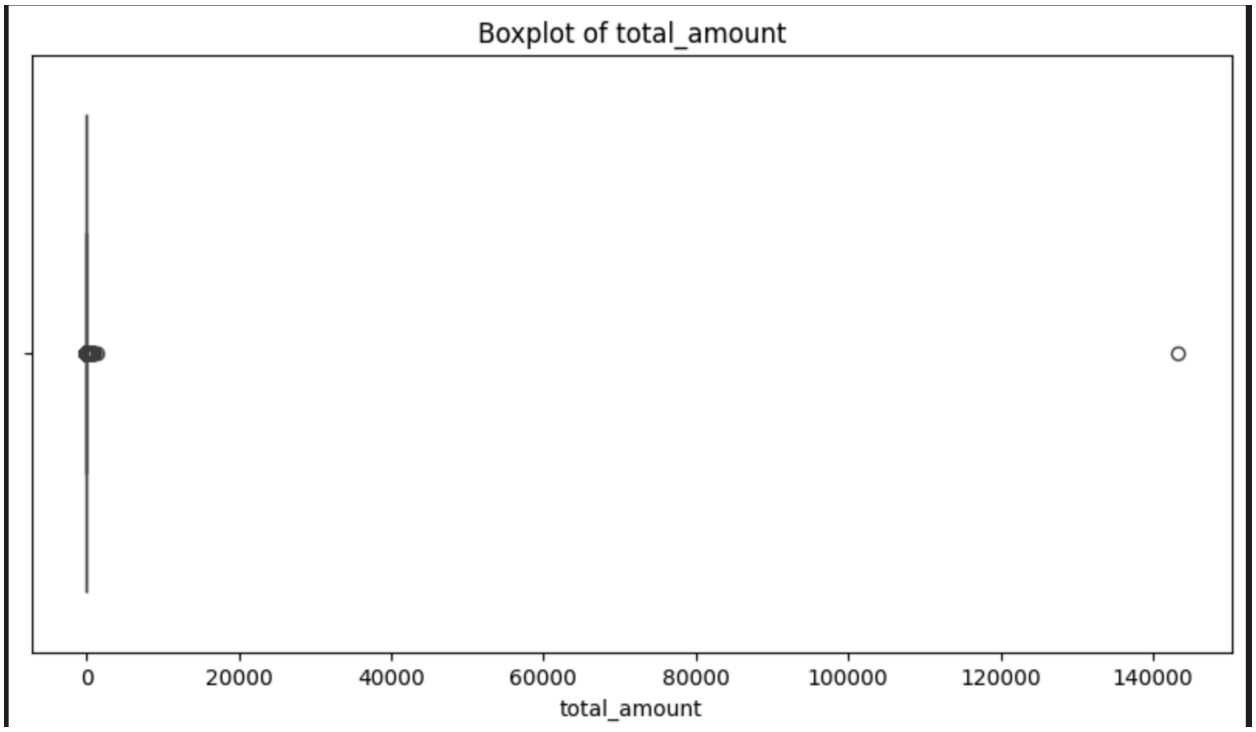


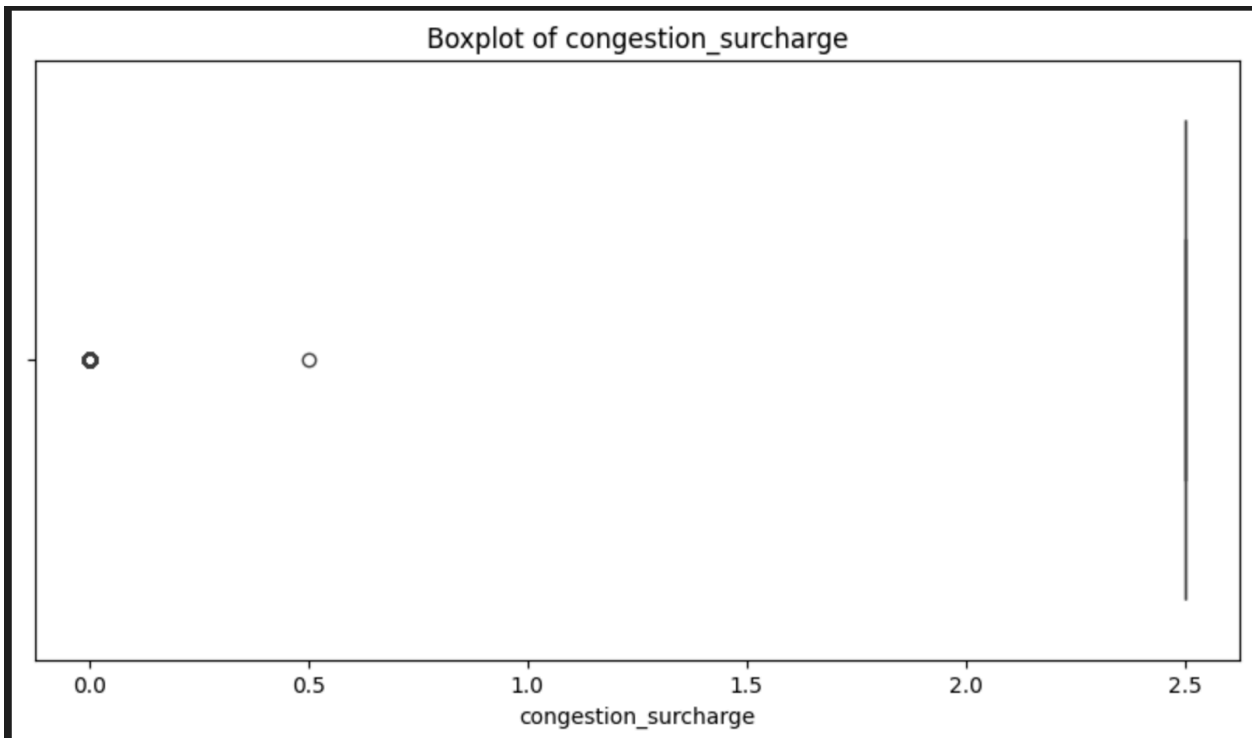
Cisco Confidential



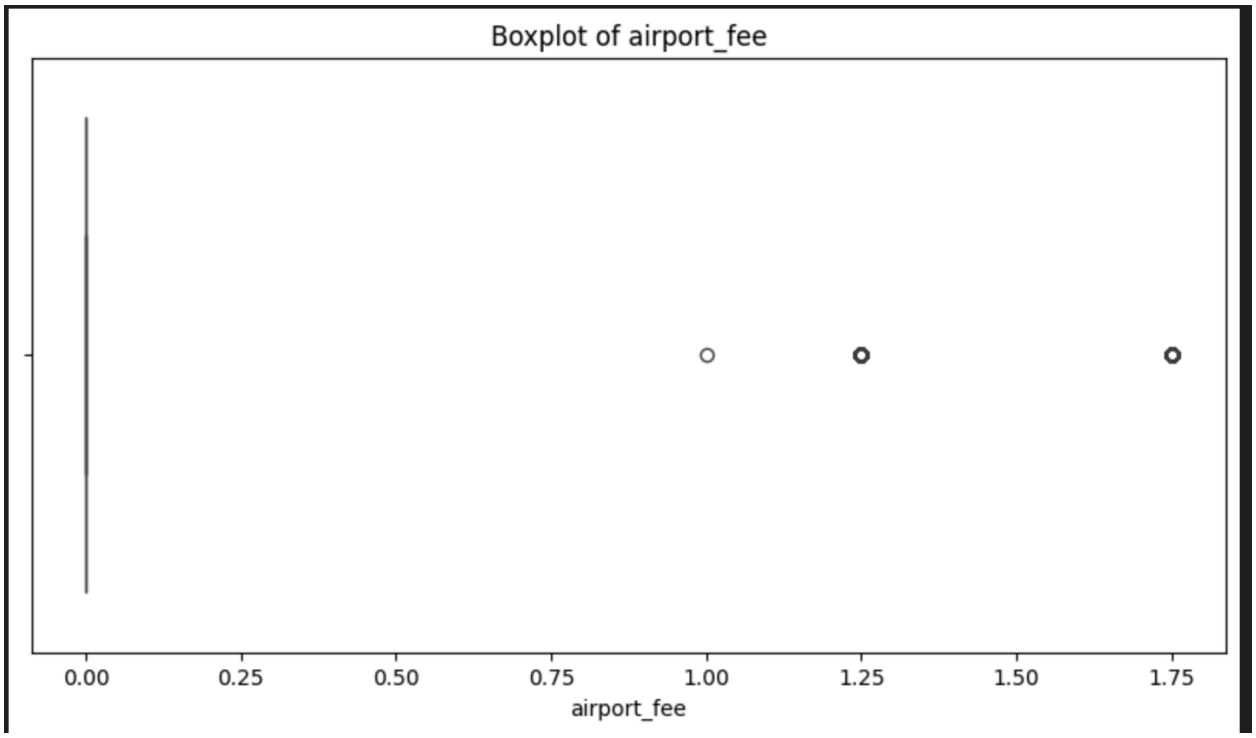


Cisco Confidential





Cisco Confidential



2.3.1. Check outliers in payment type, trip distance and tip amount columns

a. I removed trips with very high fares for very short distances, trips over 250 miles, zero fare and distance between different zones, and invalid payment types. This helps prevent incorrect records from affecting analysis.

```
# remove passenger_count > 6
df = df[df['passenger_count'] <= 6].copy()
```

```
# Continue with outlier handling
near_zero_trip_dist_and_high_fare = ((df["trip_distance"] <= 0.1) & (df["fare_amount"]
> 300))

zero_distance_fare_different_zones = (
    (df["trip_distance"] == 0)
    & (df["fare_amount"] == 0)
    & (df["PULocationID"] != df["DOLocationID"])
)
```

Cisco Confidential

```
excessive_distance = df["trip_distance"] > 250
invalid_payment_type = df["payment_type"] == 0
negative_distance = df["trip_distance"] < 0

# Combine the conditions so each row is removed only once
outlier_mask = (
    near_zero_trip_dist_and_high_fare
    | zero_distance_fare_different_zones
    | excessive_distance
    | invalid_payment_type
    | negative_distance
)

print(f"Outlier rows to remove: {outlier_mask.sum()}")
df = df.loc[~outlier_mask].copy()
df.reset_index(drop=True, inplace=True)
df.head()
# Do any columns need standardising?
#Didn't see any other column which require standardisation.
#highest_fare = df.sort_values("fare_amount", ascending=False).head(10)
#highest_fare

#Entry 1486624 has high fare amount 143163.45, this looks like an error. Removing this
entry
df = df[df.index != 1486624].copy()
df.reset_index(drop=True, inplace=True)

# Show relationship between fare and trip duration
df["tpep_dropoff_datetime"] = pd.to_datetime(df["tpep_dropoff_datetime"])
df["tpep_pickup_datetime"] = pd.to_datetime(df["tpep_pickup_datetime"])
df["trip_duration"] = (df["tpep_dropoff_datetime"] -
df["tpep_pickup_datetime"]).dt.total_seconds() / 60  # duration in minutes

#Check entries which have trip_duration more than 2000 minutes
#df[df["trip_duration"] > 2000]

#Enteris with more than 2000 minutes looks wrong enteries.
df = df[df["trip_duration"] <= 2000].copy()
df.reset_index(drop=True, inplace=True)
```

Cisco Confidential

3. Exploratory Data Analysis

3.1. General EDA: Finding Patterns and Trends

3.1.1. Classify variables into categorical and numerical

a. I classified VendorID, RatecodeID, PULocationID, DOLocationID, and payment_type as categorical because they represent groups or labels.

b. Pickup and drop-off datetimes are temporal,

c. while passenger_count, trip_distance, trip_duration, and all fare and surcharge amounts are numerical.

d. pickup_hour is classified as categorical because it has finite number of values from 1 to 24.

```
#VendorID: Categorical
#tpep_pickup_datetime: Temporal
#tpep_dropoff_datetime: Temporal
#passenger_count: Numerical
#trip_distance: Numerical
#RatecodeID: Categorical
#PULocationID: Categorical
#DOLocationID: Categorical
#payment_type: Categorical
#pickup_hour: Categorical
#trip_distance: Numerical

#The monetary variables (fare_amount, extra, mta_tax, tip_amount, tolls_amount,
improvement_surcharge, total_amount, congestion_surcharge, and airport_fee) are
numerical variables.
```

3.1.2. Analyse the distribution of taxi pickups by hours, days of the week, and months

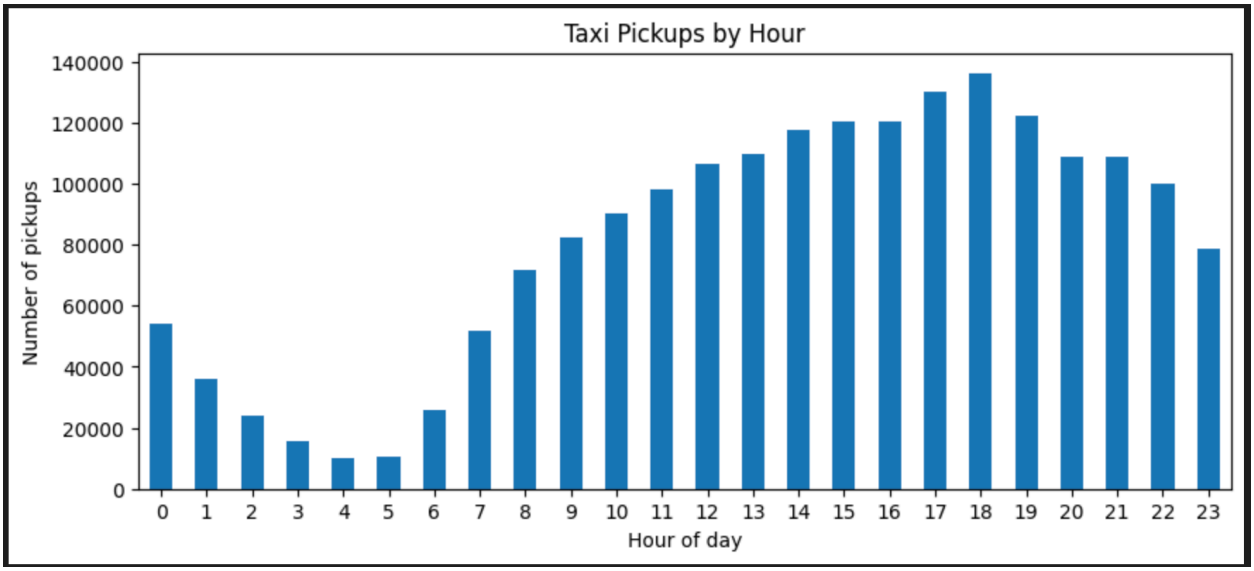
I analysed taxi pickups by hour, day of the week and month using the pickup datetime.

- The results show that pickups are generally busiest in the evening and lowest around 5 a.m.
- In weekday demand is generally higher during the day and on weekend demand is more at night.

Cisco Confidential

- Monthly pickup counts also vary, showing that demand changes throughout the year.

```
- # Find and show the hourly trends in taxi pickups
- pickup_time = pd.to_datetime(df["tpep_pickup_datetime"])
- hourly_pickups =
  pickup_time.dt.hour.value_counts().reindex(range(24),
  fill_value=0)
-
- hourly_pickups.plot(kind="bar", figsize=(10, 4), title="Taxi
  Pickups by Hour")
- plt.xlabel("Hour of day")
- plt.ylabel("Number of pickups")
- plt.xticks(range(24), rotation=0)
- plt.show()
```

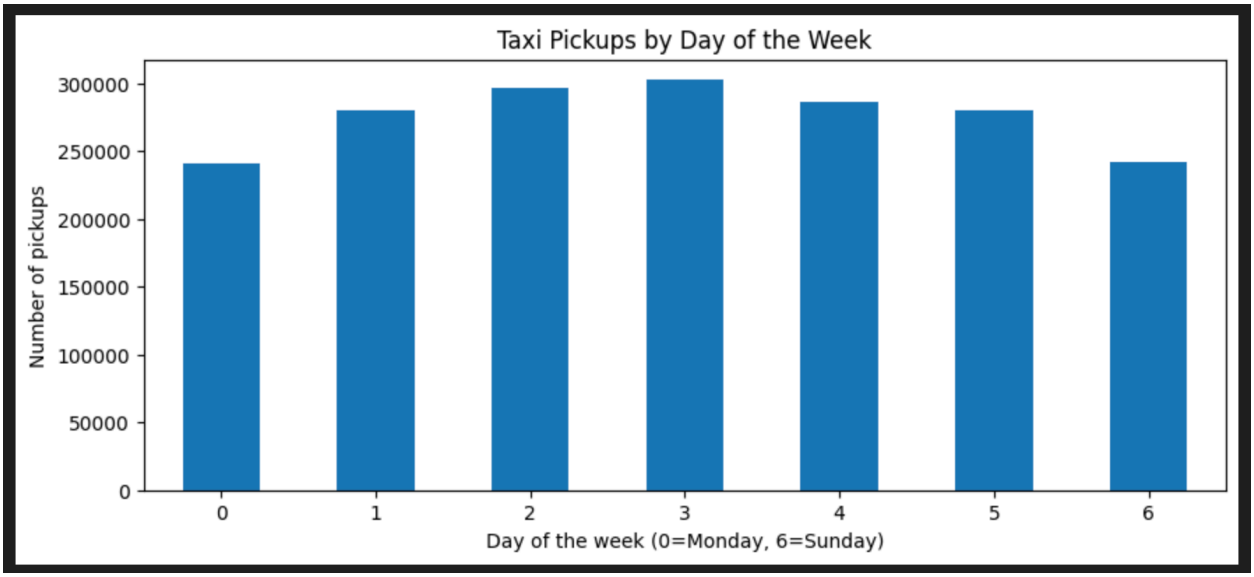



```
# Find and show the daily trends in taxi pickups (days of the week)
pickup_time = pd.to_datetime(df["tpep_pickup_datetime"])
daily_pickups = pickup_time.dt.dayofweek.value_counts().reindex(range(7),
fill_value=0)

daily_pickups.plot(kind="bar", figsize=(10, 4), title="Taxi Pickups by Day of the
Week")
```

Cisco Confidential

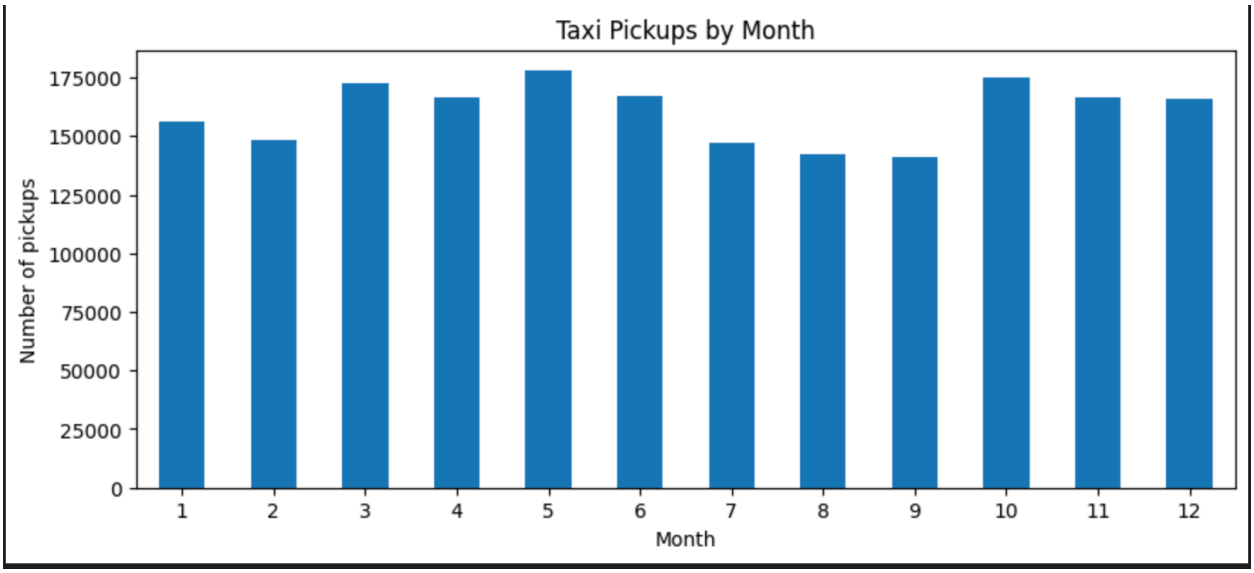
```
plt.xlabel("Day of the week (0=Monday, 6=Sunday)")
plt.ylabel("Number of pickups")
plt.xticks(range(7), rotation=0)
plt.show()
```




```
# Show the monthly trends in pickups
pickup_time = pd.to_datetime(df["tpep_pickup_datetime"])
monthly_pickups = pickup_time.dt.month.value_counts().reindex(range(1, 13),
fill_value=0)

monthly_pickups.plot(kind="bar", figsize=(10, 4), title="Taxi Pickups by Month")
plt.xlabel("Month")
plt.ylabel("Number of pickups")
plt.xticks(range(12), rotation=0)
plt.show()
```

Cisco Confidential



3.1.3. Filter out the zero/negative values in fares, distance and tips

I created a separate `analysis_df` containing trips with positive `fare_amount` and `total_amount`, so zero-value fares and totals do not affect the financial analysis. I kept trips with zero `trip_distance`, as these may be valid same-zone or cancelled trips.

```
# Create a df with non zero entries for the selected parameters.
analysis_df = df[
    (df["fare_amount"] > 0) &
    (df["total_amount"] > 0)
].copy()


max_fare = analysis_df["fare_amount"].max()
display(analysis_df.loc[analysis_df["fare_amount"] == max_fare])
```

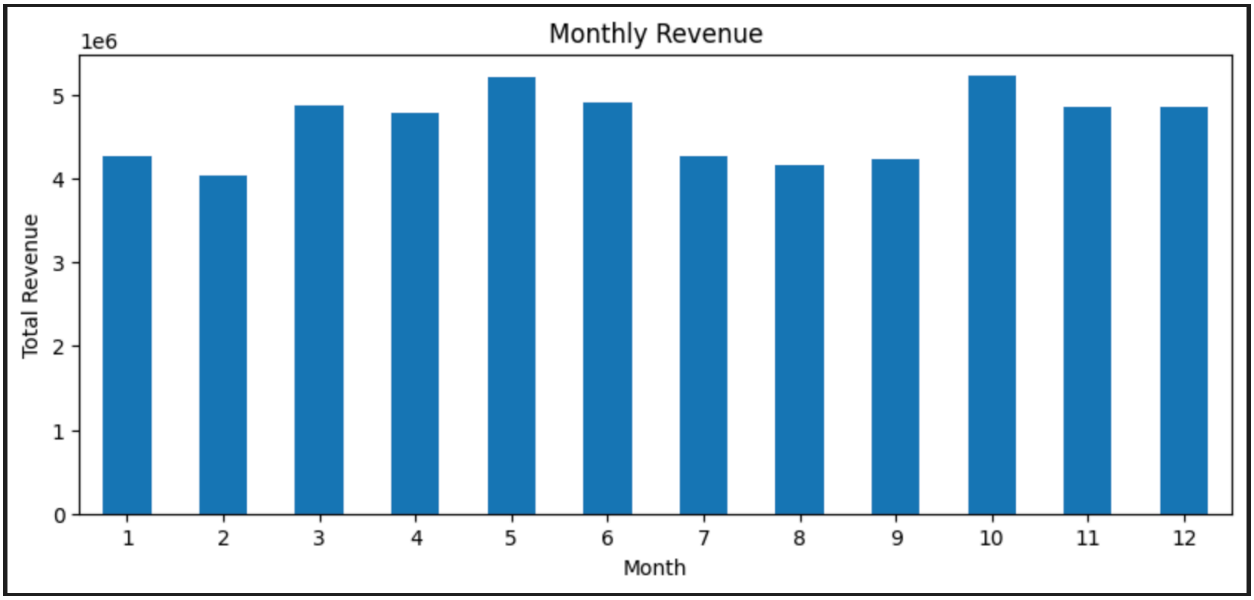
3.1.4. Analyse the monthly revenue trends

The chart shows that revenue varies across the year. Highest revenue was earned at October month while lowest revenue was earned in February month

Cisco Confidential

```
# Group data by month and analyse monthly revenue
pickup_time = pd.to_datetime(analysis_df["tpep_pickup_datetime"])
monthly_revenue = analysis_df.groupby(pickup_time.dt.month)["total_amount"].sum()

monthly_revenue.plot(kind="bar", figsize=(10, 4), title="Monthly Revenue")
plt.xlabel("Month")
plt.ylabel("Total Revenue")
plt.xticks(range(12), rotation=0)
plt.show()
```



3.1.5. Find the proportion of each quarter's revenue in the yearly revenue

I calculated each quarter's share by dividing its total revenue by the yearly revenue. The results show that in 2024 Q4 highest revenue is earned and 2023 Q3 revenue was lowest

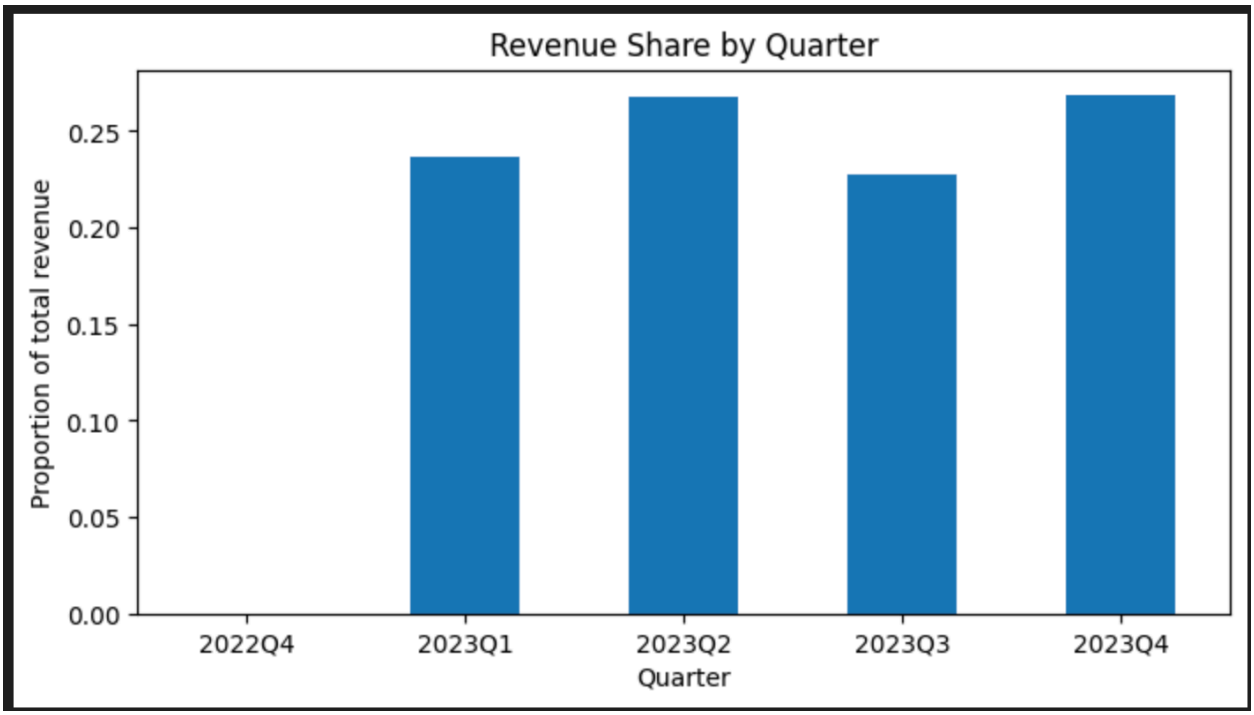
```
# Calculate proportion of each quarter
analysis_df["pickup_time"] = pd.to_datetime(analysis_df["tpep_pickup_datetime"])
analysis_df["quarter"] = analysis_df["pickup_time"].dt.to_period("Q")
```

Cisco Confidential

```
quarterly_revenue = analysis_df.groupby("quarter")["total_amount"].sum()
quarterly_revenue_prop = quarterly_revenue / quarterly_revenue.sum()

display(quarterly_revenue_prop)

quarterly_revenue_prop.plot(kind="bar", figsize=(8, 4), title="Revenue Share by
Quarter")
plt.xlabel("Quarter")
plt.ylabel("Proportion of total revenue")
plt.xticks(rotation=0)
plt.show()
```



3.1.6. Analyse and visualise the relationship between distance and fare amount

Fare amount generally increase with distance.

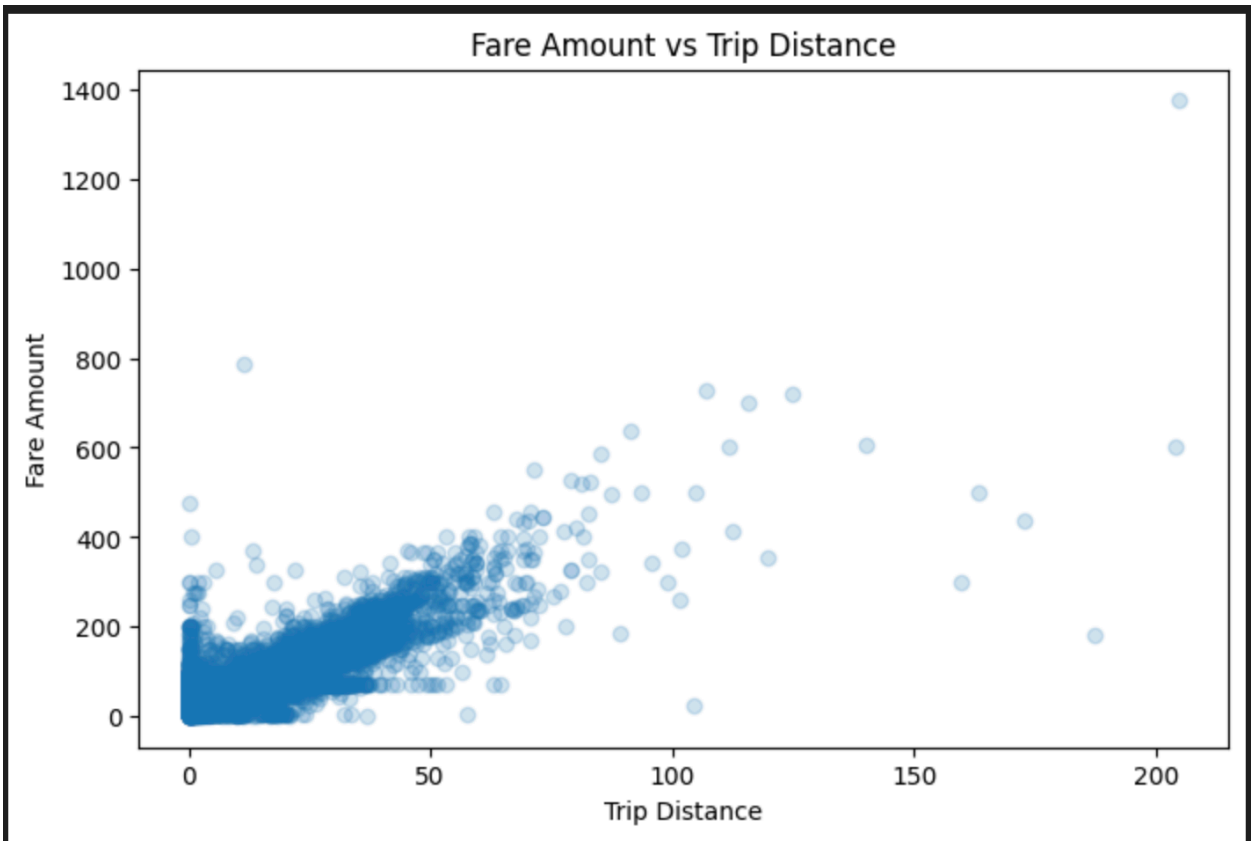
```
# Show how trip fare is affected by distance
distance_fare = analysis_df[analysis_df["trip_distance"] > 0][["trip_distance",
"fare_amount"]].copy()
```

Cisco Confidential

```python
# Scatter plot
plt.figure(figsize=(8, 5))
plt.scatter(distance_fare["trip_distance"], distance_fare["fare_amount"], alpha=0.2)
plt.title("Fare Amount vs Trip Distance")
plt.xlabel("Trip Distance")
plt.ylabel("Fare Amount")
plt.show()

# Correlation
corr = distance_fare["trip_distance"].corr(distance_fare["fare_amount"])
print("Correlation between trip_distance and fare_amount:", corr)

max_fair = distance_fare["fare_amount"].max()
print("Maximum fare amount:", max_fair)
```



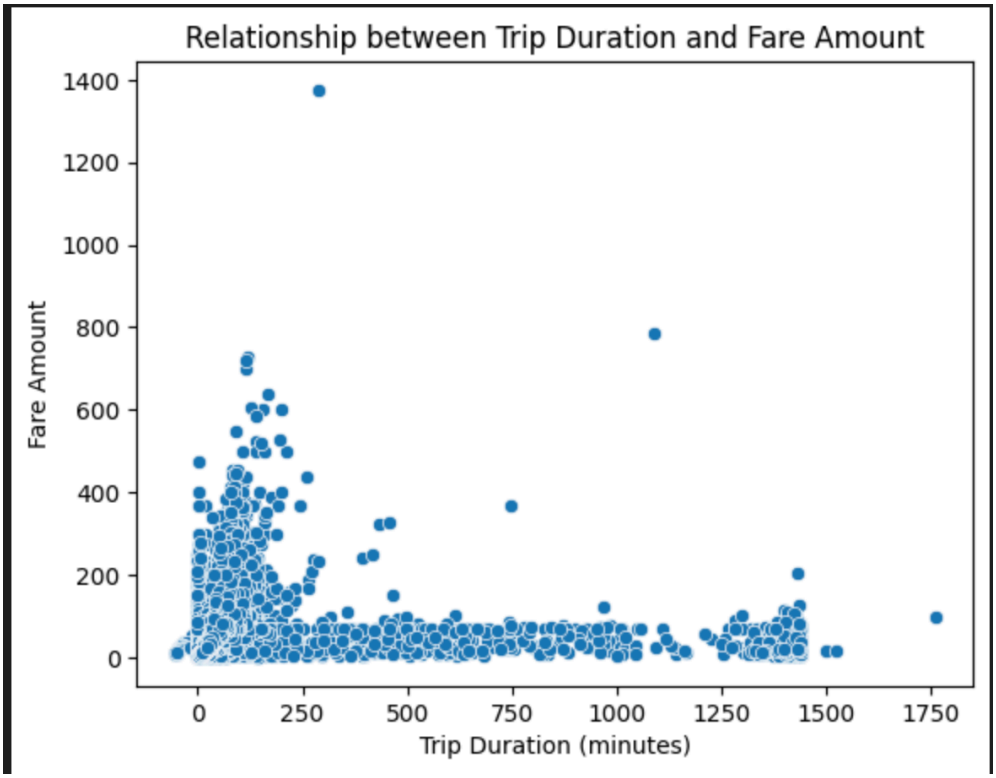
3.1.7. Analyse the relationship between fare/tips and trips/passengers

The scatter plot does not show a clear relationship between fare amount and trip duration. Fares vary widely for trips of similar duration, suggesting that duration alone does not explain the fare.

Cisco Confidential

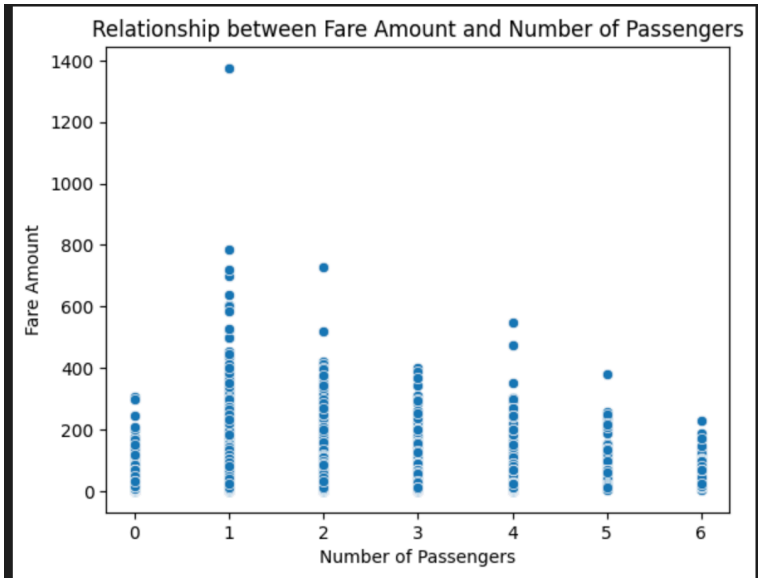
There is not much clear relationship between fare amount and passenger count.

```
# Show relationship between fare and trip duration
analysis_df["tpep_dropoff_datetime"] =
pd.to_datetime(analysis_df["tpep_dropoff_datetime"])
analysis_df["tpep_pickup_datetime"] =
pd.to_datetime(analysis_df["tpep_pickup_datetime"])
analysis_df["trip_duration"] = (analysis_df["tpep_dropoff_datetime"] -
analysis_df["tpep_pickup_datetime"]).dt.total_seconds() / 60  # duration in minutes
sns.scatterplot(x="trip_duration", y="fare_amount", data=analysis_df)
plt.xlabel("Trip Duration (minutes)")
plt.ylabel("Fare Amount")
plt.title("Relationship between Trip Duration and Fare Amount")
plt.show()
```



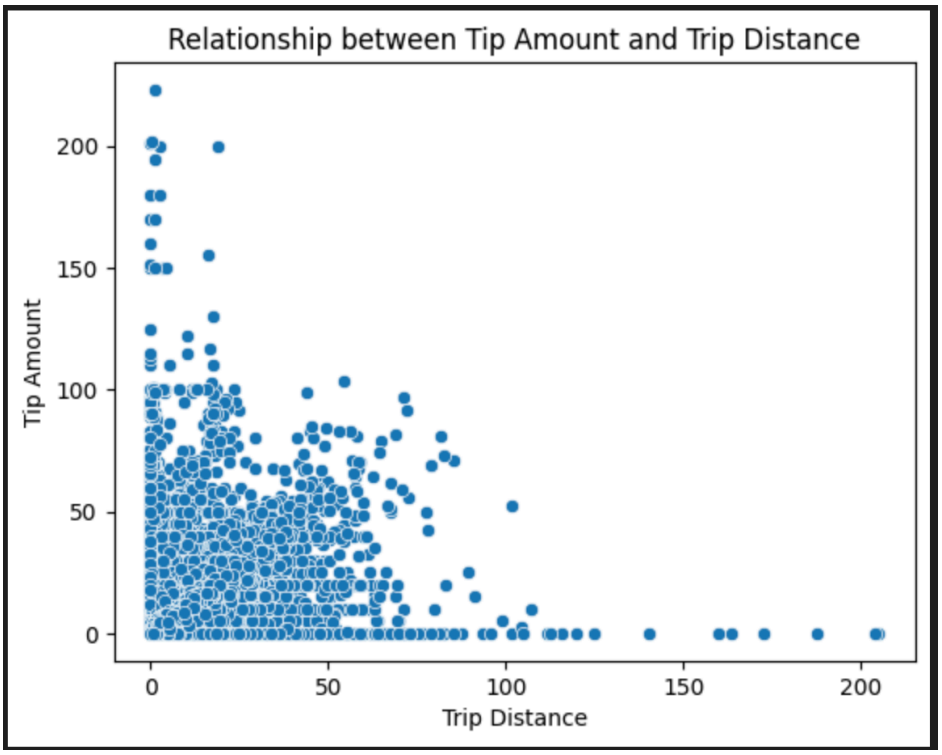
Cisco Confidential

```
# Show relationship between fare and number of passengers
sns.scatterplot(x='passenger_count', y='fare_amount', data=analysis_df)
plt.title('Relationship between Fare Amount and Number of Passengers')
plt.xlabel('Number of Passengers')
plt.ylabel('Fare Amount')
plt.show()
```




```
# Show relationship between tip and trip distance
sns.scatterplot(x='trip_distance', y='tip_amount', data=df)
plt.title('Relationship between Tip Amount and Trip Distance')
plt.xlabel('Trip Distance')
plt.ylabel('Tip Amount')
plt.show()
```

Cisco Confidential



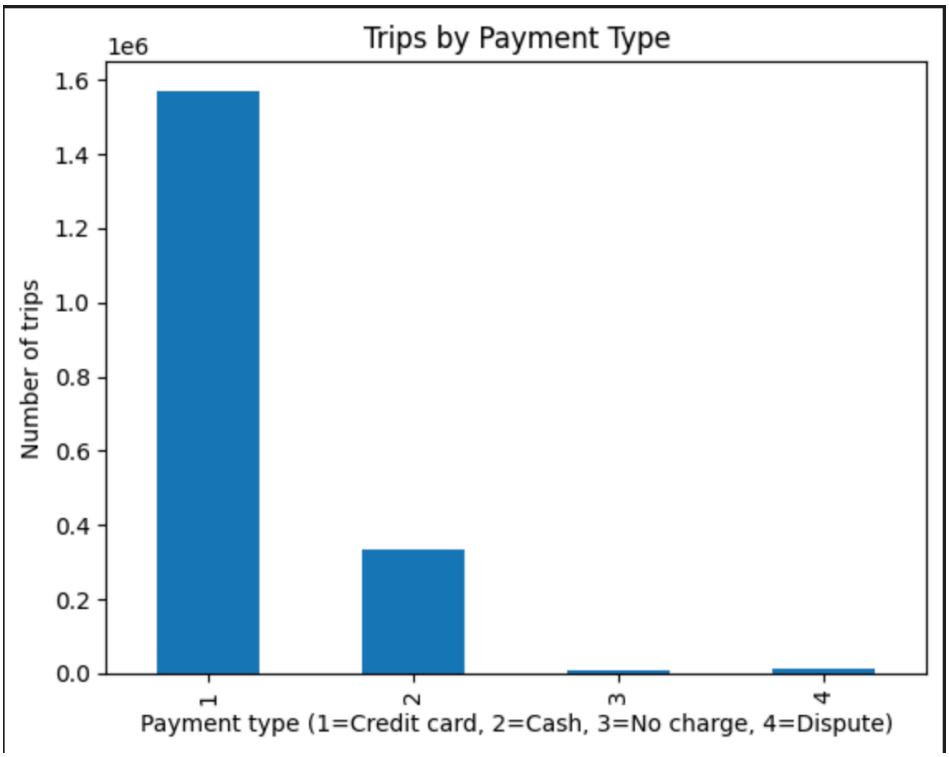
3.1.8. Analyse the distribution of different payment types

The payment type distribution shows that credit card (type 1) is the most common payment method, followed by cash (type 2). No-charge (type 3) and disputed (type 4) trips occur much less frequently. This suggests that most sampled passengers paid by credit card.

```
# Analyse the distribution of different payment types (payment_type).
payment_counts = df["payment_type"].value_counts().sort_index()

payment_counts.plot(kind="bar", title="Trips by Payment Type")
plt.xlabel("Payment type (1=Credit card, 2=Cash, 3=No charge, 4=Dispute)")
plt.ylabel("Number of trips")
plt.show()
```

Cisco Confidential



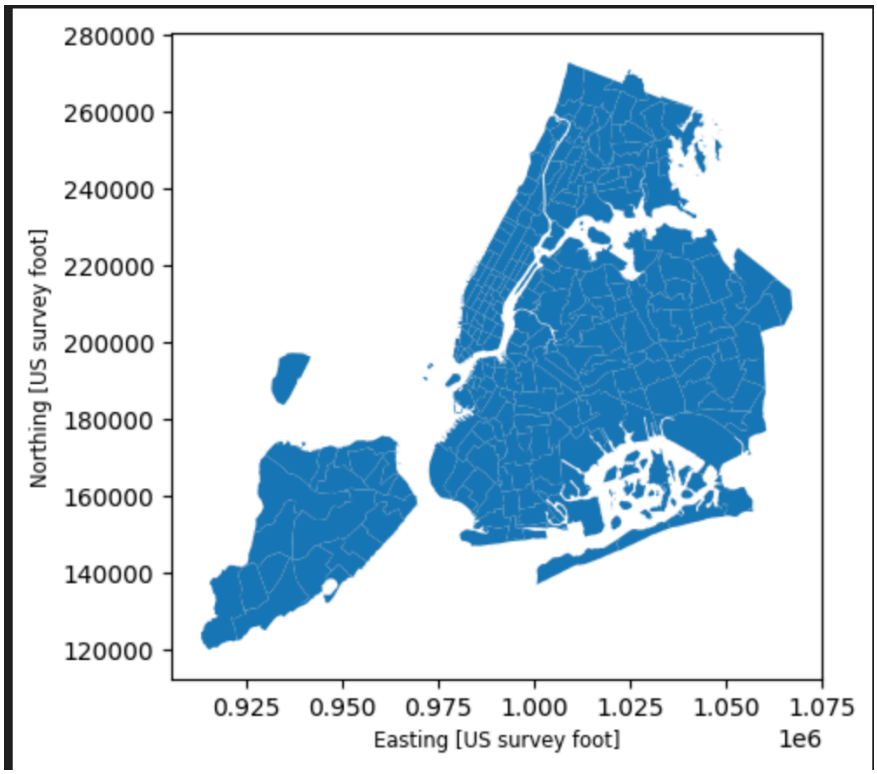
3.1.9. Load the taxi zones shapefile and display it

These are the columns printed after loading data- ['OBJECTID', 'Shape_Leng', 'Shape_Area', 'zone', 'LocationID', 'borough', 'geometry']

```
import geopandas as gpd

# Read the shapefile using geopandas
zone_info = gpd.read_file("/Users/inirala/Documents/Job Lookup/AIML LJMU/2. Data Analysis and Exploration'/EDA_assignment/Datasets and Dictionary/taxi_zones/taxi_zones.shp")  # read the .shp file using gpd
print(zone_info.columns.tolist())
print(zone_info.info())
zone_info.plot()
```

Cisco Confidential



3.1.10. Merge the zone data with trips data

Merged analysis_df with zone_info with LocationID

```
# Merge zones and trip records using locationID and PULocationID
analysis_df_merged = analysis_df.merge(
    zone_info,
    left_on="PULocationID",
    right_on="LocationID",
    how="left"
)
```

Cisco Confidential

3.1.11. Find the number of trips for each zone/location ID

We are doing groupby("PULocationID") to find number of trips for each pickup location:

```
# Group data by location and calculate the number of trips
trip_counts =
analysis_df_merged.groupby("PULocationID").size().reset_index(name="trip_count")
trip_counts
```

3.1.12. Add the number of trips for each zone to the zones dataframe

In 3.1.11 step counted pickups for each `PULocationID`. This code matches those counts to the corresponding zone's `LocationID` and dataframes are merged.

```
# Merge trip counts back to the zones GeoDataFrame
zones_with_trips = zone_info.merge(
    trip_counts,
    left_on="LocationID",
    right_on="PULocationID",
    how="left"
)

zones_with_trips["trip_count"] = zones_with_trips["trip_count"].fillna(0).astype(int)
zones_with_trips.head()
```

3.1.13. Plot a map of the zones showing number of trips

Here we are creating a map showing how many sampled taxi pickups started in each zone.

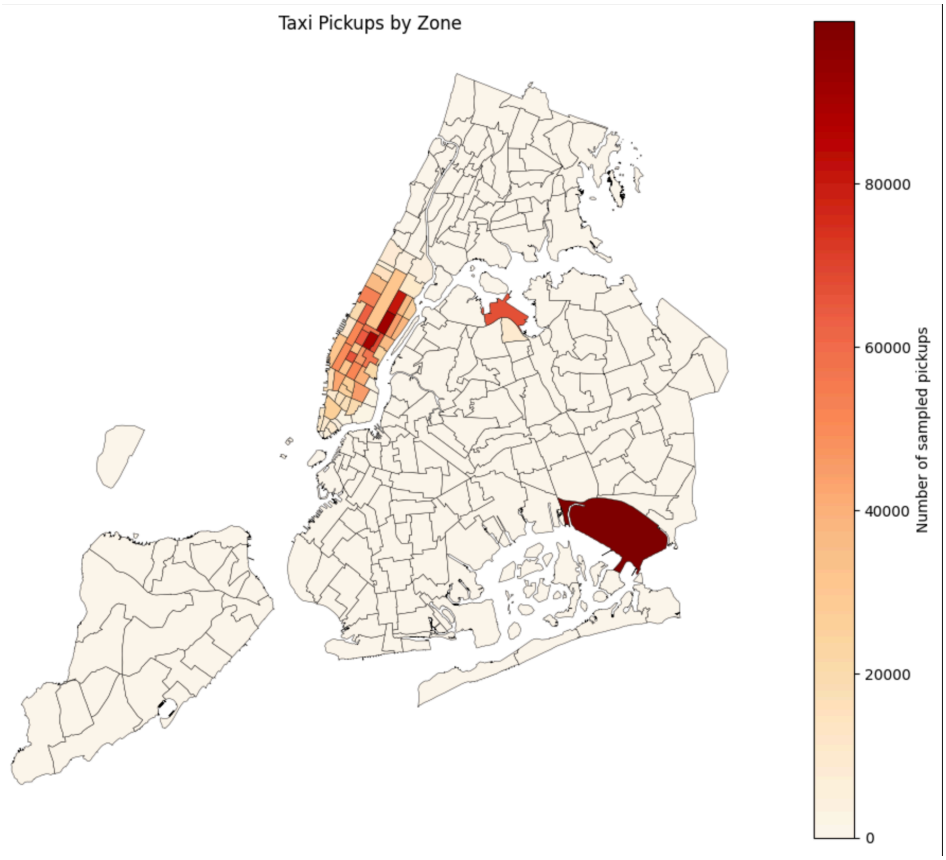
Cisco Confidential

```
# Define figure and axis
fig, ax = plt.subplots(1, 1, figsize=(12, 10))

# Plot the map and display it
zones_with_trips.plot(
    column="trip_count",
    cmap="OrRd",
    linewidth=0.3,
    edgecolor="black",
    legend=True,
    legend_kwds={"label": "Number of sampled pickups", "orientation": "vertical"},
    ax=ax
)

ax.set_title("Taxi Pickups by Zone")
ax.set_axis_off()
plt.show()
```

Cisco Confidential




```
# can you try displaying the zones DF sorted by the number of trips?
zones_with_trips.sort_values("trip_count", ascending=False)[
    ["LocationID", "zone", "borough", "trip_count"]
]
```

3.1.14. Conclude with results

The analysis shows that taxi demand changes by time and location. Pickups are higher in the evening and lowest around 5 AM. Weekday demand is generally higher during the day, while weekend demand is higher at night. Fares usually increase when trip distance or duration increases. The map also shows that some zones have more pickups than others. The company can use these patterns to plan driver shifts and place more taxis in busy zones.

Cisco Confidential

3.2. Detailed EDA: Insights and Strategies

3.2.1. Identify slow routes by comparing average speeds on different routes

It leaves out trips with zero or negative distance or duration, then groups the remaining trips by pickup hour, starting zone, and ending zone. It calculates the average distance and duration for each route, then estimates its speed in miles per hour.

Finally, it shows the route with the lowest estimated speed for each hour. This can help identify routes where traffic may be slow.

```
trips = analysis_df_merged[
    (analysis_df_merged["trip_duration"] > 0)
    & (analysis_df_merged["trip_distance"] > 0)
].copy()

trips["pickup_hour"] = pd.to_datetime(trips["tpep_pickup_datetime"]).dt.hour

routes = (
    trips.groupby(["pickup_hour", "PULocationID", "DOLocationID"])
    .agg(
        avg_distance=("trip_distance", "mean"),
        avg_duration=("trip_duration", "mean"),
    )
)

routes["speed_mph"] = routes["avg_distance"] / (routes["avg_duration"] / 60)

slowest_routes = routes.loc[
    routes.groupby("pickup_hour")["speed_mph"].idxmin()
]

slowest_routes
```

3.2.2. Calculate the hourly number of trips and identify the busy hours

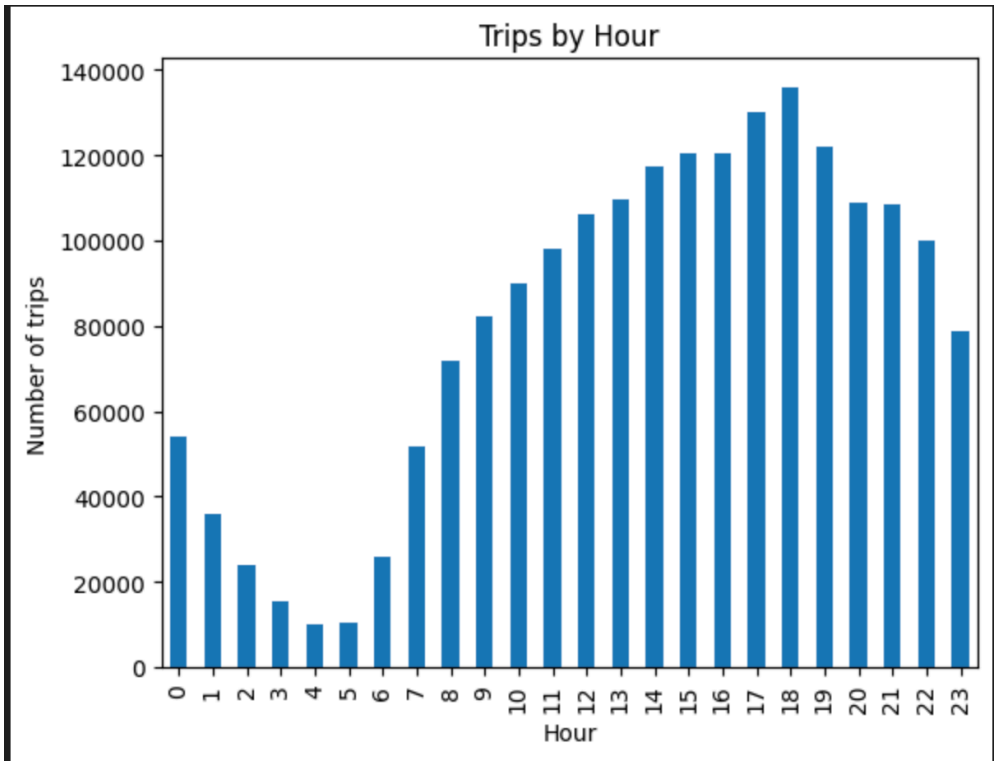
Here we are counting taxi trips for each hour of the day and then showing them in a bar chart. We are then printing the busiest hour and how many sampled trips happened during that hour.

Cisco Confidential

```python
# Visualise the number of trips per hour and find the busiest hour
hours = pd.to_datetime(df["tpep_pickup_datetime"]).dt.hour
hourly_trips = hours.value_counts().reindex(range(24), fill_value=0)

hourly_trips.plot(kind="bar", title="Trips by Hour")
plt.xlabel("Hour")
plt.ylabel("Number of trips")
plt.show()

print("Busiest hour:", hourly_trips.idxmax())
print("Number of trips:", hourly_trips.max())
```



Cisco Confidential

3.2.3. Scale up the number of trips from above to find the actual number of trips

In **3.2.3**, I find the five busiest hours in my sample. Since I sampled 5% of the trips, I divide each count by 0.05 to estimate the actual number of trips for those hours. These numbers are estimates, not exact totals.

```
# Scale up the number of trips

# Fill in the value of your sampling fraction and use that to scale up the numbers
sample_fraction = 0.05

top_5_sampled = hourly_trips.nlargest(5)
top_5_estimated = (top_5_sampled / sample_fraction).round().astype(int)

top_5_results = pd.DataFrame({
    "sampled_trips": top_5_sampled,
    "estimated_actual_trips": top_5_estimated
})
top_5_results.index.name = "pickup_hour"

display(top_5_results)
```

3.2.4. Compare hourly traffic on weekdays and weekends

Here we are comparing the average number of taxi pickups each hour on weekdays and weekends. It groups the trips by day type and hour, then shows the results in a line chart.

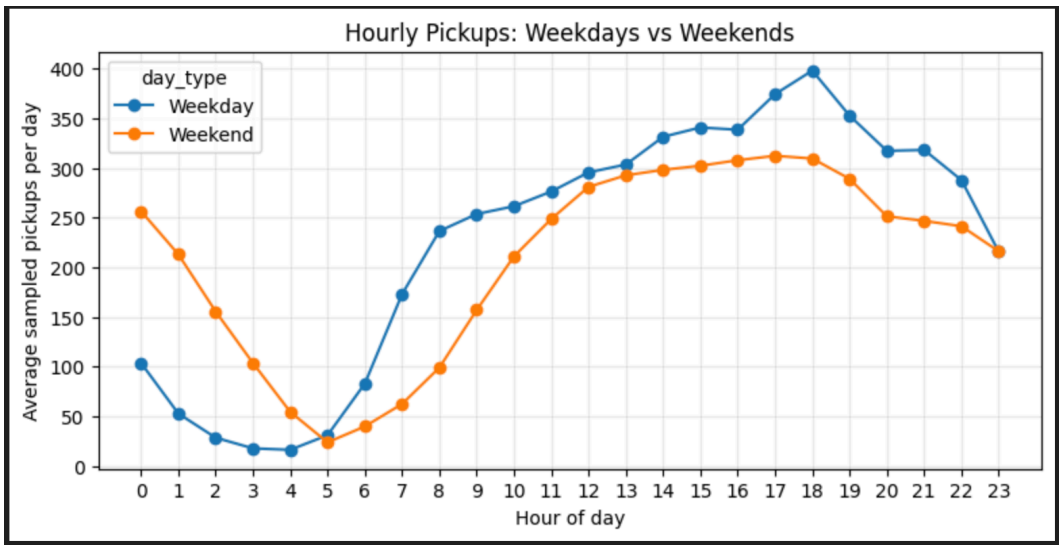
Cisco Confidential

```
# Compare traffic trends for the week days and weekends
pickup_time = pd.to_datetime(df["tpep_pickup_datetime"])

traffic = pd.DataFrame({
    "date": pickup_time.dt.date,
    "hour": pickup_time.dt.hour,
    "day_type": np.where(pickup_time.dt.dayofweek < 5, "Weekday", "Weekend")
})

hourly_counts = traffic.groupby(["day_type", "hour"]).size().unstack(fill_value=0)
days = traffic.groupby("day_type")["date"].nunique()
average_per_day = hourly_counts.div(days, axis=0)

average_per_day.T.plot(kind="line", marker="o", figsize=(9, 4))
plt.xlabel("Hour of day")
plt.ylabel("Average sampled pickups per day")
plt.title("Hourly Pickups: Weekdays vs Weekends")
plt.xticks(range(24))
plt.grid(True, alpha=0.3)
plt.show()
```



3.2.5. Identify the top 10 zones with high hourly pickups and drops

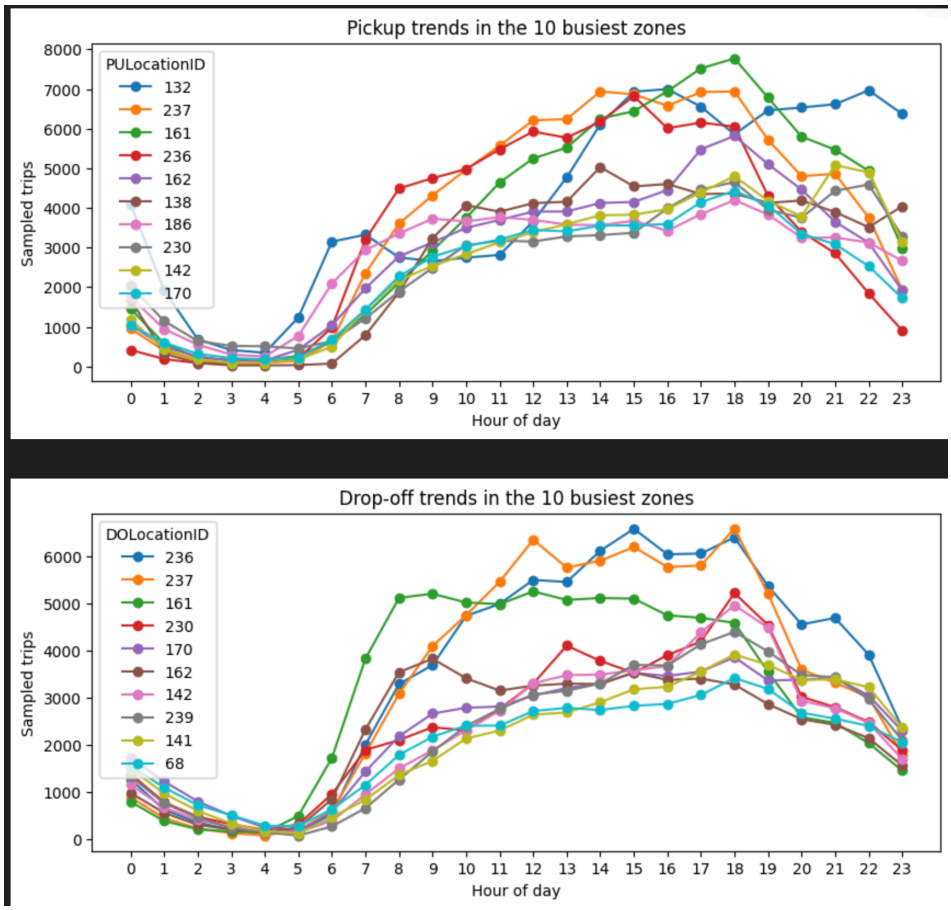
In this code we are finding the 10 busiest pickup zones and the 10 busiest drop-off zones. We are then showing a chart of how trips in those zones change throughout the day.

Cisco Confidential

```python
# Find top 10 pickup and dropoff zones
for zone_col, time_col, label in [
    ("PULocationID", "tpep_pickup_datetime", "Pickup"),
    ("DOLocationID", "tpep_dropoff_datetime", "Drop-off")
]:
    data = analysis_df_merged[[zone_col, time_col]].copy()
    data["hour"] = pd.to_datetime(data[time_col]).dt.hour
    hourly = data.groupby([zone_col, "hour"]).size().unstack(fill_value=0)
    hourly = hourly.reindex(columns=range(24), fill_value=0)
    top10 = hourly.sum(axis=1).nlargest(10).index

    hourly.loc[top10].T.plot(figsize=(10, 4), marker="o",
                             title=f"{label} trends in the 10 busiest zones")
    plt.xlabel("Hour of day")
    plt.ylabel("Sampled trips")
    plt.xticks(range(24))
    plt.show()
```

Cisco Confidential



3.2.6. Find the ratio of pickups and dropoffs in each zone

This code counts the pickups and drop-offs in each zone, then shows the 10 zones with the highest pickup-to-drop-off ratios and the 10 with the lowest ratios. A high ratio means more pickups than drop-offs; a low ratio means fewer pickups than drop-offs.

```
# Find the top 10 and bottom 10 pickup/dropoff ratios
pickups = analysis_df_merged["PULocationID"].value_counts()
dropoffs = analysis_df_merged["DOLocationID"].value_counts()

ratio = (pickups / dropoffs).dropna().rename("pickup_dropoff_ratio")
display(ratio.nlargest(10))
display(ratio.nsmallest(10))
```

Cisco Confidential

3.2.7. Identify the top zones with high traffic during night hours

This code finds the 10 busiest pickup zones and the 10 busiest drop-off zones between 11 PM and 5 AM. It uses the pickup time for pickup zones and the drop-off time for drop-off zones.

```
# During night hours (11pm to 5am) find the top 10 pickup and dropoff zones
# Note that the top zones should be of night hours and not the overall top zones

for zone, time in [("PULocationID", "tpep_pickup_datetime"),
                   ("DOLocationID", "tpep_dropoff_datetime")]:
    night = analysis_df_merged[pd.to_datetime(
        analysis_df_merged[time]).dt.hour.isin([23, 0, 1, 2, 3, 4])]
    display(night[zone].value_counts().head(10))
```

3.2.8. Find the revenue share for nighttime and daytime hours

This code calculates the share of total revenue earned during night hours (11 PM to 5 AM) and daytime hours. It uses the pickup time to decide whether each trip is a night or day trip.

```
# Filter for night hours (11 PM to 5 AM)
hours = pd.to_datetime(analysis_df_merged["tpep_pickup_datetime"]).dt.hour
revenue = analysis_df_merged.groupby(hours.isin([23, 0, 1, 2, 3,
4]))["total_amount"].sum()
display(revenue.rename({True: "Night", False: "Day"}) / revenue.sum())
```

3.2.9. For the different passenger counts, find the average fare per mile per passenger

Cisco Confidential

This code calculates the average fare per mile for each passenger, grouped by the number of passengers in the taxi. It ignores trips where the passenger count is zero.

```
# Analyse the fare per mile per passenger for different passenger counts
x = analysis_df[analysis_df["passenger_count"] > 0]
(x["fare_amount"] / x["trip_distance"] /
x["passenger_count"]).groupby(x["passenger_count"]).mean()
```

3.2.10. Find the average fare per mile by hours of the day and by days of the week

This code calculates the average fare per mile for each hour of the day and each day of the week. It helps compare how the fare per mile changes over time.

```
# Compare the average fare per mile for different days and for different times of the
day
fare_per_mile = analysis_df["fare_amount"] / analysis_df["trip_distance"]
pickup = pd.to_datetime(analysis_df["tpep_pickup_datetime"])

display(fare_per_mile.groupby(pickup.dt.hour).mean().rename("Fare per mile by hour"))
display(fare_per_mile.groupby(pickup.dt.day_name()).mean().rename("Fare per mile by
day"))
```

3.2.11. Analyse the average fare per mile for the different vendors

This code calculates the average fare per mile for each taxi company during each hour of the day. It displays the results in a table so the companies can be compared.

Cisco Confidential

```
# Compare fare per mile for different vendors
display(fare_per_mile.groupby([analysis_df["VendorID"], pickup.dt.hour])
        .mean().unstack().round(2))
```

3.2.12. Compare the fare rates of different vendors in a distance-tiered fashion

This code groups trips into three distance ranges: up to 2 miles, 2–5 miles, and over 5 miles. It then compares the vendors' average fare per mile for each range.

```
# Defining distance tiers
tiers = pd.cut(analysis_df["trip_distance"], [0, 2, 5, float("inf")],
               labels=["Up to 2 miles", "2–5 miles", "Over 5 miles"])

display(fare_per_mile.groupby([analysis_df["VendorID"], tiers],
                              observed=True).mean().unstack().round(2))
```

3.2.13. Analyse the tip percentages

This code calculates the tip as a percentage of the fare. Then it compares the average tip percentage by trip distance, number of passengers, and pickup hour.

```
#  Analyze tip percentages based on distances, passenger counts and pickup times
tip_pct = 100 * analysis_df["tip_amount"] / analysis_df["fare_amount"]
```

Cisco Confidential

```
distance_tier = pd.cut(analysis_df["trip_distance"], [0, 2, 5, float("inf")],
                       labels=["Up to 2 mi", "2–5 mi", "Over 5 mi"])

display(tip_pct.groupby(distance_tier, observed=True).mean().round(2))
display(tip_pct.groupby(analysis_df["passenger_count"]).mean().round(2))
display(tip_pct.groupby(pickup.dt.hour).mean().round(2))
```

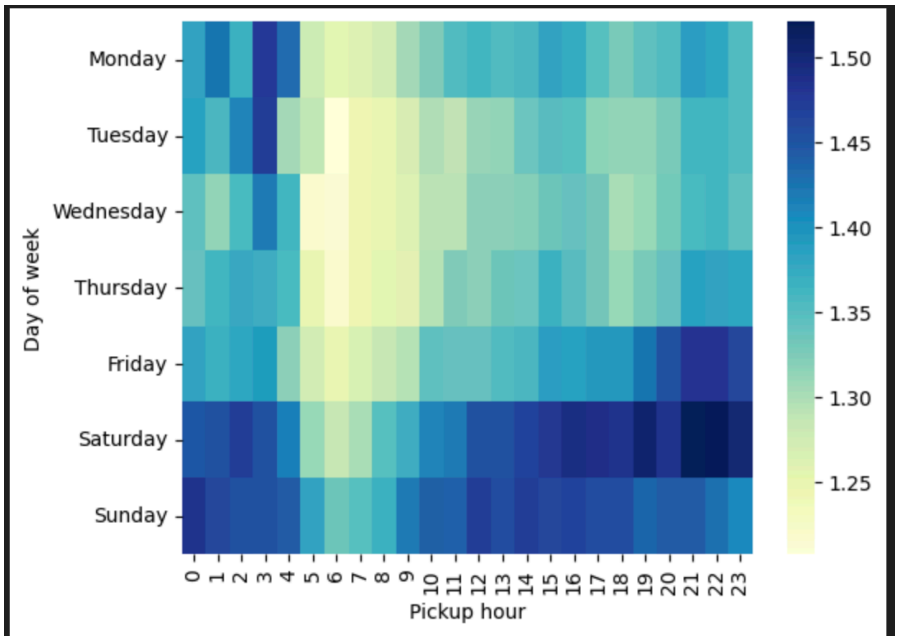
3.2.14. Analyse the trends in passenger count

This code calculates the average number of passengers for each hour and day of the week. It shows the results in a heatmap, so it's easy to compare them.

```
# See how passenger count varies across hours and days
passenger_time = analysis_df.assign(
    day=pickup.dt.day_name(), hour=pickup.dt.hour
).pivot_table(index="day", columns="hour", values="passenger_count", aggfunc="mean")

passenger_time = passenger_time.reindex(
    ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"]
)
sns.heatmap(passenger_time, cmap="YlGnBu")
plt.xlabel("Pickup hour")
plt.ylabel("Day of week")
plt.show()
```

Cisco Confidential



3.2.15. Analyse the variation of passenger counts across zones

3.2.16. Analyse the pickup/dropoff zones or times when extra charges are applied more frequently.

4. Conclusions

4.1. Final Insights and Recommendations

4.1.1. Recommendations to optimize routing and dispatching based on demand patterns and operational inefficiencies.

The analysis shows that taxi demand changes by hour and day. Pickups are higher in the evening, while demand is lowest around 5 AM. Weekday and weekend patterns are also different. The company should schedule more taxis during busy hours and fewer during quiet hours.

Cisco Confidential

Drivers should use alternative routes when the analysis shows that a route is slow. This can reduce waiting time and help drivers complete more trips.

4.1.2. Suggestions on strategically positioning cabs across different zones to make best use of insights uncovered by analysing trip trends across time, days and months.

The pickup analysis shows that some zones have more demand than others. More taxis should be placed near the busiest pickup zones, especially before busy hours. During the night, taxis should be positioned near the zones with the most night pickups. The company should also watch the pickup and drop-off balance in each zone. If many taxis drop passengers in a zone but few passengers start trips there, taxis may need to move to a busier nearby zone.

4.1.3. Propose data-driven adjustments to the pricing strategy to maximize revenue while maintaining competitive rates with other vendors.

The analysis shows that fare amount generally increases with trip distance and duration. Average fare per mile can also differ by hour, day, distance, and vendor. The company should use these results to review its pricing and compare its rates with other providers

Cisco Confidential